

Full Length Article

Dynamic graph representation learning based on multi-scale attention

Pingtao Duan^{a,b}, Yanqun Wu^{a,b}, Jiayi Wei^{a,b}, Jing Tian^{a,b}, Jiang Wu^{a,b},
Lin Lin^{a,c}, Huilai Miao^{d,e}, Jinchao Cui^a, Hui Wang^f, Dong Ouyang^{a,b,*}, Bo Jin^{a,b,g,*}

^a School of Biomedical Engineering, Guangdong Medical University, Dongguan, 523808, China

^b Guangdong Medical University Health and Medical Big Data Engineering Center, Guangdong Medical University, Dongguan, 523808, China

^c Key Laboratory of Medical Electronics and Medical Imaging Equipment, Dongguan, 523808, China

^d Zhanjiang Key Laboratory for Diagnosis and Repair of Digestive System Organ Injuries, Zhanjiang, 524023, China

^e Department of Hepatobiliary and Pancreatic Surgery, The Second Affiliated Hospital of Guangdong Medical University, Zhanjiang, 524023, China

^f Dalian Institute of Chemical Physics, Chinese Academy of Sciences, Dalian, 116023, China

^g Dalian University of Technology, Dalian, 116024, China

ARTICLE INFO

Keywords:

Dynamic graph

Hypergraph

Representation learning

Multi-scale attention

ABSTRACT

Dynamic graph representation learning aims to learn informative and predictive node representations from evolving graph structures and node attributes. Although existing methods achieve promising results for modeling structural evolution and temporal dependencies, they often emphasize only a subset of the relevant structural or temporal information and lack a unified framework to jointly extract local topology, high-order structural similarity, node attribute dynamics, and multi-scale temporal dependencies. To fill this gap, we propose a Multi-scale Attention-based Dynamic Graph Network (MADGN) integrating structure-aware modeling and hierarchical attention. First, MADGN builds a time-aware hypergraph with structural descriptors and a temporal decay function to capture high-order correlations between structurally similar non-adjacent nodes. Next, Local Structure Attention and Temporal Feature Attention separately model fine-grained neighborhood topology and time-varying node attributes. We fuse these complementary embeddings via Long-Term Attention Fusion to capture long-range spatiotemporal evolutionary patterns, and design a dual-objective loss to jointly preserve connective proximity and structural similarity in the embedding space. Experiments on four real-world dynamic graph datasets validate that MADGN outperforms state-of-the-art baselines on dynamic link prediction, dynamic new link prediction, and node classification tasks. Our model achieves up to 6.17% relative AUC improvement for dynamic new link prediction, as well as maximum 14.5% Accuracy (ACC) and 7.9% AUC gains for node classification, while retaining efficient training overhead.

1. Introduction

With the increasing complexity of real-world systems, such as social networks, biological processes, and transportation networks, graphs have emerged as a fundamental data structure for effectively representing complex structural information (Newman, 2003; Xu et al., 2020; Zheng et al., 2025). Graph representation learning is a key technology for extracting latent patterns from graph-structured data, aiming to map nodes into low-dimensional embeddings while preserving both structural topologies and node attributes (Kazemi et al., 2020). Since these learned representations serve as the foundation for a wide range of downstream inference tasks, graph representation learning has con-

sequently become a prominent research focus in the fields of complex system modeling and data mining (Newman, 2003; Zheng et al., 2025).

Early research on graph representation learning primarily relied on the assumption that the topological structures and interaction relationships within graph data remain invariant during the modeling process (Zheng et al., 2025). Researchers proposed various representation learning methods to extract structural information from static graphs. Early random walk-based methods learned node representations by generating node sequences and drawing upon concepts from context modeling (Grover & Leskovec, 2016; Perozzi et al., 2014); subsequently, matrix factorization methods mapped structural similarities between nodes into a low-dimensional space by decomposing adjacency or transition

* Corresponding authors.

E-mail addresses: pingtaoduan@gdmu.edu.cn (P. Duan), yqwu00@gdmu.edu.cn (Y. Wu), weijiayi@gdmu.edu.cn (J. Wei), tianjing1@gdmu.edu.cn (J. Tian), wujiang@gdmu.edu.cn (J. Wu), linlin_bme@gdmu.edu.cn (L. Lin), miaohl-gdwk@gdmu.edu.cn (H. Miao), cjcw@163.com (J. Cui), hwang1606@dicp.ac.cn (H. Wang), ouyangdong@gdmu.edu.cn (D. Ouyang), jibo@gdmu.edu.cn (B. Jin).

<https://doi.org/10.1016/j.neunet.2026.109603>

Received 16 March 2026; Received in revised form 21 July 2026; Accepted 4 September 2026

Available online 11 September 2026

0893-6080/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

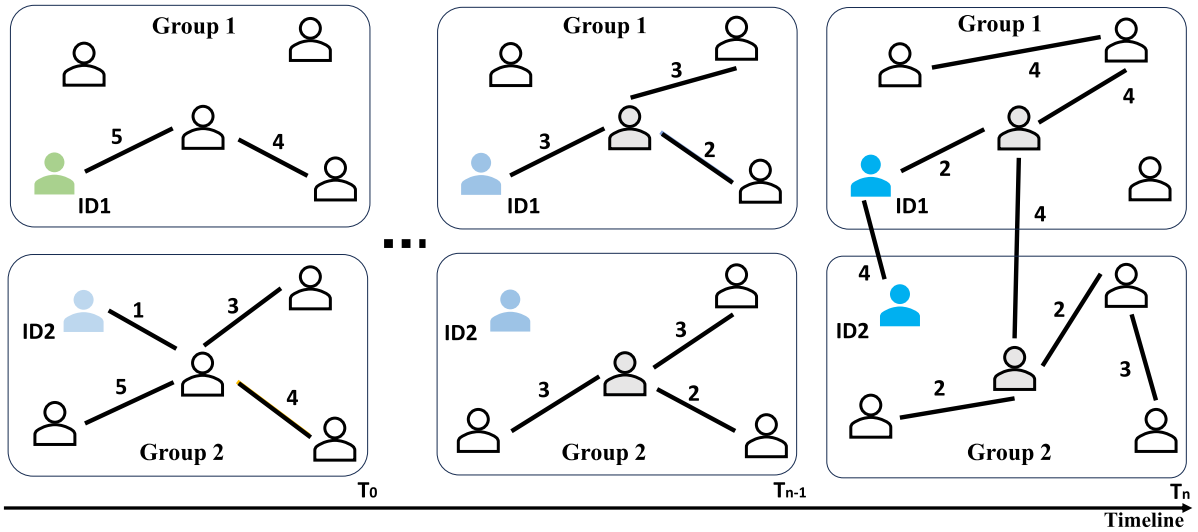


Fig. 1. Multi-scale evolution of a dynamic social network from T_0 to T_n . Node colors indicate attribute states, edge weights represent interaction intensity, and gray nodes illustrate structurally similar roles that develop cross-group connections over time.

matrices (Cao et al., 2015). With the rise of deep learning, graph convolutional neural networks achieved joint modeling of graph structures and node attributes through neighborhood feature propagation and aggregation (Kipf & Welling, 2017). However, such methods typically rely on pairwise connections between nodes, making it difficult to directly characterize higher-order associations involving multiple nodes. To address this issue, hypergraph neural networks introduced hyperedges capable of connecting multiple nodes simultaneously, thereby enhancing the ability to model group relationships and non-pairwise interactions (Feng et al., 2019). Building on this, attention mechanisms further improved the models' capacity to adaptively weigh the importance of different structural relationships. For instance, GAT (Veličković et al., 2018) adaptively aggregates local neighborhood information by assigning learnable attention weights to different neighbors, while HGAT (Bai et al., 2021) extends the attention mechanism to hypergraphs, establishing adaptive information interaction between nodes and hyperedges to capture more complex higher-order structural relationships. Although these models have achieved remarkable results in mining static graph information, they are difficult to adapt to the evolution of nodes, edges and interaction relationships in complex systems, and they cannot fully capture structural changes and their temporal dependencies, thus limiting their applicability in real-world scenarios (Zheng et al., 2025).

Addressing the inherent limitations of static graphs, dynamic graph representation learning, capable of simultaneously modeling temporal dependencies and structural evolution features, has become an active research area in graph learning. Methods based on recurrent neural networks (Gao et al., 2023; Goyal et al., 2020; Pareja et al., 2020) capture the evolution of structural information over time by generating node representations across graph snapshots through a combination of temporal modeling and graph convolution-based aggregation. Although effective at modeling sequential evolution, these methods may incur sequential computational overhead and primarily rely on local pairwise aggregation, failing to explicitly model high-order information regarding non-neighboring nodes or long-range temporal dependencies. Meanwhile, models employing attention mechanisms to capture structural evolution can adaptively focus on significant changes but still fail to explicitly model the interactions between dynamically changing node attributes and high-order structural features (Sankar et al., 2020; Xu et al., 2020). To further explore higher-order structural information, some methods have introduced higher-order topological units such as k-core structures (Liu et al., 2022), motifs (Liu et al., 2021), or

role-based hypergraphs (Du et al., 2024) to break through local neighborhood constraints and effectively preserve the global structure and higher-order features of graph data. However, such methods struggle to balance local topological features with high-order structural similarity within a unified representation space, nor do they effectively resolve the challenges of modeling dynamic node attribute evolution and long-term temporal changes in graph data. In summary, while existing dynamic graph representation learning methods capture evolutionary characteristics from various modeling dimensions, individual methods typically capture only partial information, resulting in node embeddings that may not preserve complementary structural and temporal information (Khoshraftar & An, 2024). An ideal dynamic graph node representation must simultaneously preserve local connective proximity, the latent high-order structural similarity between non-adjacent nodes, the co-evolutionary patterns of node attributes and graph topology, and temporal dependencies across both short- and long-term scales (Ndunguru et al., 2025).

To intuitively illustrate the necessity of modeling multi-scale spatiotemporal information collaboratively, this paper analyzes a dynamic social network as an example, as shown in Fig. 1. This schematic represents network entities as nodes and user interactions at different time steps as dynamic edges, while using Group 1 and Group 2 to demonstrate the evolution of network structure and node states over time. The dynamic characteristics are primarily manifested in three aspects. First, the colors of nodes ID1 and ID2 change over time, reflecting the dynamic evolution of node attributes. In real-world systems, user preferences, behavioral states, and entity functions typically change over time. Concurrently, interactions between nodes with increasingly similar attributes intensify, indicating a close correlation between attribute changes and topological evolution: changes in node states are often associated with edge formation and interaction intensity, while structural changes, in turn, affect node representations. Second, the number, density, and weight of edges within a community fluctuate over time. These structural changes, which include the strength of short-term relationships and long-term interaction trends, are the core basis for characterizing the temporal dynamics of a network. Finally, gray nodes occupying similar topological positions or structural roles across the two communities gradually form inter-community connections, revealing multi-scale structural dependencies within the dynamic graph. This phenomenon suggests that node evolution is influenced not only by local neighborhood interactions but also by cross-community structural similarities and global topological relationships.

Investigating such high-order structural associations helps explain complex evolutionary behaviors in real-world networks, such as cross-community information propagation, user migration, and system structural reorganization (Ndunguru et al., 2025). In summary, the evolution of dynamic graphs is driven by a combination of changing node attributes, short-term interaction fluctuations, long-term relationship trends, and both local and global structural dependencies. Consequently, modeling from only a single structural or temporal scale may lead to the loss of dynamic information and representation bias, thereby compromising the reliability of downstream tasks such as prediction, inference, and decision-making.

Targeting the above research objectives, this paper proposes a multi-scale attention-based dynamic graph network (MADGN) for systematic and efficient capture of multi-dimensional spatiotemporal information in dynamic graphs, which constructs a cross-scale representation learning framework supported by four collaborative modules. The Short-Term Global Structure Attention module constructs a time-aware hypergraph based on structural similarity and temporal decay, thereby explicitly modeling high-order interactions and their spatiotemporal evolution. By leveraging the expressive capacity of hypergraphs together with an adaptive attention mechanism, this module encodes global structural correlations and identifies informative structural patterns. Complementarily, the Local Structure Attention module focuses on fine-grained topological changes within local neighborhoods and captures microscopic structural evolution through adaptive aggregation of neighborhood information. Meanwhile, the Temporal Feature Attention module explicitly models temporal variations in node attributes, enhancing the model's sensitivity to evolving feature patterns. Collectively, these three modules provide complementary views of dynamic graph evolution from the perspectives of global structure, local topology, and temporal node attributes. The resulting multidimensional representations are subsequently integrated by the Long-Term Attention Fusion module, which applies an attention mechanism to the fused representation sequence to explicitly model long-range dependencies and capture persistent structural and semantic patterns over extended temporal horizons. In addition, MADGN is optimized through connective-proximity and structural-similarity objectives, enabling the latent space to preserve both observed interactions and structural equivalence. Through the coordinated operation of these components, MADGN learns unified dynamic node representations that jointly characterize topological evolution and temporal dependencies across multiple temporal scales and structural hierarchies, while preserving local connective proximity, high-order structural similarity, dynamic node attributes, and multi-scale temporal dependencies.

In summary, the main contributions are as follows:

- We propose MADGN that jointly models local connectivity, high-order structural relations, node-attribute evolution, and multi-scale temporal dependencies within a unified representation framework. This formulation enables complementary structural and temporal information to be coordinated rather than learned through isolated modeling processes.
- MADGN introduces a novel integration of time-aware hypergraph encoding and multi-scale attention mechanisms. This mechanism facilitates the complementary modeling and integration of high-order node correlations and fine-grained pairwise interactions, allowing for a comprehensive characterization of both local and global structural variations and short- and long-term temporal dependencies.
- Extensive experiments on real-world dynamic graph datasets demonstrate consistent improvements over representative baselines in dynamic link prediction, dynamic new link prediction, and node classification. Additional ablation, representation quality, parameter sensitivity, visualization, and efficiency analyses further validate the contributions of the proposed components.

2. Related work

2.1. Graph representation learning

The primary objective of graph representation learning is to encode graph-structured data into low-dimensional representations while preserving essential topological and semantic information. Early graph representation learning methods primarily relied on random walks and matrix factorization. DeepWalk (Perozzi et al., 2014) and node2vec (Grover & Leskovec, 2016) learned node representations from random-walk-generated contexts using the Skip-gram objective, whereas GraRep (Cao et al., 2015) modeled high-order network proximity through matrix factorization of graph-derived similarity matrices.

With the development of deep learning, graph representation learning has increasingly adopted neural message-passing architectures. Graph neural networks (GNNs) use message-passing mechanisms to integrate node attributes with graph topology. GCN (Kipf & Welling, 2017) aggregates information from local neighborhoods through graph convolution, GraphSAGE (Hamilton et al., 2017) employs neighborhood sampling to support inductive learning, and GAT (Veličković et al., 2018) assigns adaptive attention weights to neighboring nodes. However, these models primarily capture local pairwise dependencies and do not explicitly represent multi-node high-order interactions. Hypergraph-based methods generalize pairwise relations to hyperedges that can connect multiple nodes, enabling the modeling of non-pairwise and high-order interactions. HGCN (Feng et al., 2019) models high-order interactions through hypergraph convolution. HGAT (Bai et al., 2021) and HyperGAT (Ding et al., 2020) introduce attention mechanisms into hypergraph representation learning, whereas HyperSAGE (Arya et al., 2020) develops an inductive aggregation strategy for hypergraphs.

2.2. Dynamic graph representation learning

Dynamic graph representation learning aims to learn informative and predictive node representations from graph structures that evolve over time. Existing methods generally approach dynamic graph modeling through temporal evolution, structural preservation, or specialized dynamic mechanisms.

A major line of research combines graph neural networks with recurrent architectures to model temporal dependencies across snapshots. DGCN (Gao et al., 2023) employs LSTMs to evolve GCN parameters across graph snapshots, thereby modeling temporal structural changes. Similarly, DynAERNN (Goyal et al., 2020) combines autoencoders with recurrent neural networks to learn temporally evolving node embeddings. In contrast, EvolveGCN (Pareja et al., 2020) updates GCN parameters through recurrent units rather than directly evolving node embeddings, allowing the graph encoder to adapt across snapshots. GCRN (Seo et al., 2018) combines graph convolution for spatial aggregation with recurrent networks for temporal modeling. Although these methods effectively model sequential dependencies, their structural aggregation is primarily based on local pairwise message passing or recurrent parameter updates. Consequently, for nodes with similar structures but no direct connection, this type of method lacks the ability to model higher-order structural information.

Another line of research focuses on high-order proximity or global structural similarity within dynamic graphs. CTGCN (Liu et al., 2022) uses k-core-based temporal graph convolution to preserve local connective proximity and global structural similarity during graph evolution. MTSN (Liu et al., 2021) captures high-order structural patterns and node attributes through a motif encoder while modeling temporal dynamics using temporal shift operations. RTGCN (Du et al., 2024) uses role-based hypergraph convolution to capture global structural-role information in dynamic graphs. Collectively, these methods highlight the importance of structural roles, high-order proximity, and motif patterns in dynamic graph representation learning. However, these methods generally emphasize particular structural priors, while the joint modeling of

high-order structure, local topology, node-attribute evolution, and long-term temporal dependencies remains insufficiently explored.

Other studies explore specialized update mechanisms or alternative representation spaces for dynamic graph modeling. ROLAND (You et al., 2022) treats embeddings from different GNN layers as hierarchical node states and recursively updates them over time, thereby extending static GNNs to dynamic graphs. SEAN (Zhang et al., 2024) employs time-aware aggregation to reduce the influence of outdated historical information. DyCPM (Duan et al., 2024) models multi-relational dynamic graphs by distinguishing newly formed edges from high-frequency interactions. MRDGNN (Duan et al., 2023) captures relation-specific structural changes through topology update mechanisms tailored to different relation types. Another line of work employs spiking neural networks (SNNs) to model temporal graph evolution in an event-driven manner. SpikeNet (Li et al., 2023) represents dynamic graph evolution using spike trains to improve computational efficiency on large graphs, whereas SiGNN (Chen et al., 2025) introduces a temporal activation mechanism to integrate spiking dynamics with graph-based spatial aggregation. HTGN (Yang et al., 2021) embeds temporal graphs in hyperbolic space and combines hyperbolic graph neural networks with gated recurrent units to model temporal evolution and hierarchical structure.

These methods extend dynamic graph representation learning through recursive state updates, time-aware aggregation, relation-specific modeling, event-driven computation, and non-Euclidean representations. Nevertheless, each method primarily emphasizes a particular modeling mechanism, and the coordinated integration of local topology, high-order structural similarity, node-attribute evolution, and long-term temporal dependencies remains underexplored.

2.3. Attention-based dynamic graph modeling

Attention mechanisms assign adaptive weights to nodes, edges, historical states, and structural patterns. They are therefore widely used for structural aggregation and temporal dependency modeling in dynamic graph representation learning. Compared with fixed neighborhood aggregation and uniform temporal recursion, attention mechanisms can assign different weights to structural relations and historical states, allowing models to emphasize more informative dynamic patterns.

In the field of dynamic graph representation learning, DySAT (Sankar et al., 2020) is a representative snapshot-based attention model that applies structural self-attention within each snapshot and temporal self-attention across snapshots. TGAT (Xu et al., 2020) integrates temporal encoding into the graph attention mechanism, enabling the model to learn inductive temporal node representations based on interaction timestamps. GAEN (Shi et al., 2021) uses attention-based parameter evolution to preserve node similarities associated with temporal variation patterns while adapting to topological differences across graph snapshots. Furthermore, SimpleDyG (Wu et al., 2024) shows that lightweight Transformer-like architectures can model dynamic graph evolution with relatively low computational overhead. Collectively, these methods illustrate how attention mechanisms can support adaptive structural aggregation and temporal dependency modeling.

3. Problem statement

We formally define the task of dynamic graph representation learning as follows. A dynamic graph is modeled as a time-ordered sequence of static graph snapshots, denoted by $\mathcal{G} = \{G^1, G^2, \dots, G^T\}$, where T represents the total number of observed time steps. Each snapshot at time t is defined as $G^t = (V^t, E^t, X^t)$, where V^t is the set of nodes, E^t is the set of edges, and $X^t \in \mathbb{R}^{|V^t| \times D}$ represents the node attribute matrix with D features per node. The structural topology of G^t is encapsulated in the adjacency matrix $A^t \in \{0, 1\}^{|V^t| \times |V^t|}$, where $A^t_{ij} = 1$ if an interaction exists between nodes v_i and v_j at time t , and 0 otherwise.

The goal of dynamic graph representation learning is to learn a mapping function $F : \{G^1, G^2, \dots, G^T\} \rightarrow O^t$ that encodes high-dimensional structural and temporal information into low-dimensional latent embeddings $\mathcal{O} = \{O^1, O^2, \dots, O^T\} \in \mathbb{R}^{N \times T \times F}$. The representation vector $o^t_v \in O^t$ should capture the cumulative evolutionary trajectory of node v from the initial time step to the current time step t , where N denotes the number of nodes, T the total number of time steps, and F the dimensionality of the node embeddings.

High-quality dynamic graph embeddings must possess the following characteristics. First, strongly connected or frequently interacting nodes should have similar embeddings, thereby accurately preserving the graph's local topological connectivity. Second, nodes with similar topological structures should exhibit highly similar embeddings, reflecting the phenomenon in real-world networks where structurally similar nodes often display convergent behavior. Third, changes in node attributes may be associated with the formation and disappearance of edges, while the iterative evolution of the topological structure conversely influences the semantic expression of node attributes; the embeddings should capture their joint evolutionary patterns. Fourth, embeddings should remain temporally coherent under minor perturbations while remaining responsive to meaningful structural changes.

4. Methodology

As illustrated in Fig. 2, we propose the MADGN framework, a hierarchically coordinated system designed for dynamic graph sequences $\mathcal{G} = \{G^1, G^2, \dots, G^T\}$ and node attributes $\mathcal{X}$. MADGN aims to systematically encapsulate the temporal evolution and multi-level structural dependencies of nodes through a multi-scale attention mechanism, ultimately generating dynamic node embeddings $\mathcal{O} = \{O^1, O^2, \dots, O^T\}$ that are optimized for diverse downstream tasks. MADGN employs a Short-Term Global Structure Attention (STGSA) module. This module integrates a structural similarity metric with a temporal decay function to capture the evolutionary consistency of node patterns. By mapping structurally similar nodes into latent hyperedges, it constructs a time-aware hypergraph and its corresponding incidence matrices H^{cu} and H^{fu} . Via adaptive dual-level attention (at both the hyperedge and node levels), this module explicitly captures high-order interactions beyond pairwise constraints, yielding a global representation Z^{SG} that produces a global structural representation over a short history window. In parallel with the STGSA module, MADGN incorporates a Local Structure Attention (LSA) module and a Temporal Feature Attention (TFA) module to capture fine-grained spatiotemporal dynamics. Constrained by the adjacency topology, the LSA module models localized dependencies within a node's immediate neighborhood. By estimating pairwise attention weights, it preserves microscopic structural semantics to generate the local representation Z^{LS} . Simultaneously, the TFA module integrates node features with sinusoidal positional encodings to enhance temporal modeling performance. Using linear projection matrices W_Q , W_K , and W_V , it computes attention scores along the temporal dimension to capture transient fluctuation patterns, resulting in the temporal representation Z^{TF} . Finally, the Long-Term Attention Fusion (LTAF) module fuses the extracted global structural (Z^{SG}), local structural (Z^{LS}), and temporal feature (Z^{TF}) representations. By aggregating these multi-scale features over an extended temporal horizon, the model generates semantically consistent and robust node embeddings, thereby enabling a unified representation of complex system dynamics.

4.1. Short-term global structure attention

To capture global structural dependencies that transcend physical topological constraints and characterize their rapid evolution in the short term, we design a Short-Term Global Structure Attention (STGSA) module. This module constructs associations between non-adjacent nodes with similar structural features via an implicit

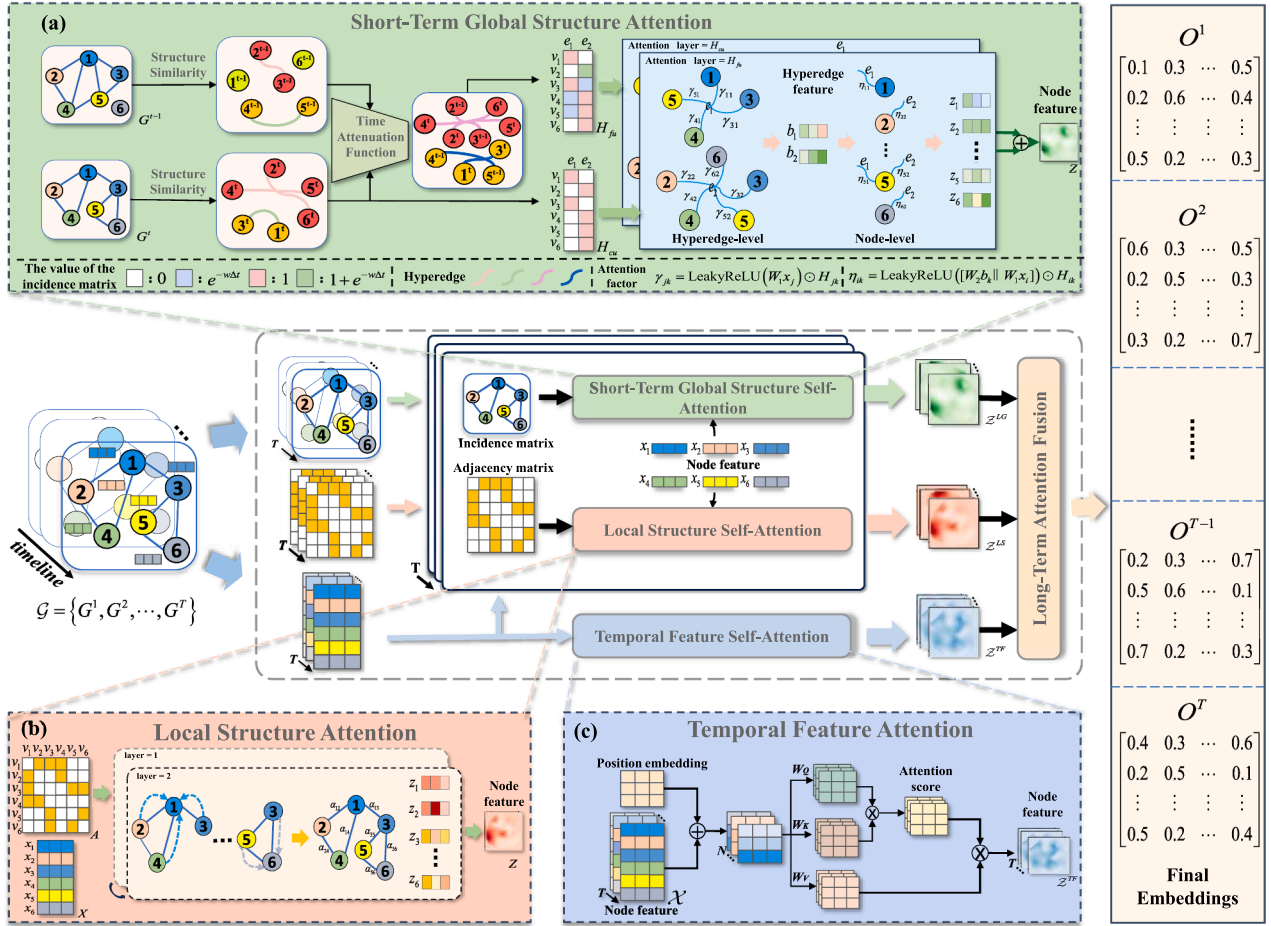


Fig. 2. Overview of MADGN. STGSA, LSA, and TFA encode complementary global structural, local topological, and temporal attribute information, which is integrated by LTA to generate dynamic node embeddings.

hypergraph and incorporates a dual-level attention mechanism to model dynamic structures.

For a graph snapshot G^t at timestamp t , the module first integrates degree-based descriptors, Weisfeiler-Lehman features, and motif statistics to extract an m -dimensional structural feature vector $\mathbf{F}_v^t = [f_1, f_2, \dots, f_m]$ for each node $v \in V$. To identify and connect nodes with similar structural contexts, we introduce a mapping function Φ . For any two nodes u and v in the graph, the following definition applies:

$$(\forall i \in 1, 2, \dots, m : f_i(u) = f_i(v)) \Rightarrow \Phi(u) \equiv \Phi(v), \quad (1)$$

which partitions the node set V into $|\mathcal{E}|$ hyperedges ($|\mathcal{E}| \leq N$) based on the consistency of structural features. Finally, the original graph snapshot G^t is projected into a hypergraph with a higher-order structure, which is formally represented by the incidence matrix $H^{cu,t} \in \mathbb{R}^{N \times |\mathcal{E}|}$, where N denotes the number of nodes and $|\mathcal{E}|$ represents the number of constructed hyperedges.

We employ a temporal decay function to nonlinearly aggregate the current structural-similarity hypergraph $H^{cu,t}$ with the topological structures of the previous k historical snapshots, constructing a fused hypergraph that combines immediacy and continuity, represented by the incidence matrix $H^{fu,t}$:

$$H^{fu,t} = H^{cu,t} + \sum_{\tau=1}^k \exp(-\omega \Delta t_{\tau}) H^{cu,t-\tau}. \quad (2)$$

Where ω is a hyperparameter representing the decay rate, and Δt_{τ} is the time span between the current timestamp t and the historical time $t - \tau$. The exponential decay term $\exp(-\omega \Delta t_{\tau})$ serves as a temporal weight allocator, ensuring that structural patterns closer to the current time

are assigned higher weights. This topological fusion enables the model to capture short-term mutations while maintaining the smoothness and consistency of structural semantics over time, enabling the model to perceive the short-term evolutionary dynamics of the structure.

For each timestamp t , after constructing the dynamic incidence matrix $H^t = \{H^{cu,t} \in \mathbb{R}^{N \times |\mathcal{E}|}, H^{fu,t} \in \mathbb{R}^{N \times |\mathcal{E}|}\}$, this module employs a dual-level attention mechanism (node-level and hyperedge-level) to model the incidence matrix and node features $X^t = [x_1, x_2, \dots, x_N] \in \mathbb{R}^{N \times D}$ with feature dimension D , thereby achieving deep coupling of node features and high-order structural semantics. This ultimately yields node embeddings $Z^{SG,t} = [z_1, z_2, \dots, z_N] \in \mathbb{R}^{N \times F}$ that integrate short-term temporal information and global structural information.

In order to acquire structure-aware hyperedge embeddings, we derive a representation $b_k \in \mathbb{R}^D$ for each hyperedge e_k by aggregating the feature information of nodes incident to it. The contribution of each node v_j to e_k is modulated by the incidence weight H_{jk} , ensuring that both structural connectivity and participation strength influence the resulting representation. Formally, the hyperedge embedding is computed as

$$b_k = \sigma \left(\sum_{v_j \in e_k} \frac{\exp(a_1^\top \gamma_{jk})}{\sum_{v_p \in e_k} \exp(a_1^\top \gamma_{pk})} W_1 x_j \right), \quad (3)$$

where $\sigma(\cdot)$ denotes a nonlinear activation function, $W_1 \in \mathbb{R}^{D \times D}$ is a learnable transformation matrix, and $a_1 \in \mathbb{R}^D$ is the hyperedge-level attention vector. The formula for computing the attention input is given by

$$\gamma_{jk} = \text{LeakyReLU}(W_1 x_j) \odot H_{jk}, \quad (4)$$

which reflects the manner in which node v_j contributes to hyperedge e_k , where H_{jk} quantifies the incidence strength and $\odot$ denotes element-wise multiplication. This design enables the module to differentiate nodes based on both feature saliency and structural relevance.

Following the extraction of all hyperedge embeddings $B = [b_1, b_2, \dots, b_K] \in \mathbb{R}^{K \times D}$, we propagate structural information back to nodes. For node v_i , let E_i denote the set of hyperedges incident to it. The updated node embedding $\tilde{x}_i$ is computed as:

$$\tilde{x}_i = \sigma \left(\sum_{e_k \in E_i} \frac{\exp(a_2^\top \eta_{ik})}{\sum_{e_p \in E_i} \exp(a_2^\top \eta_{ip})} W_2 b_k \right), \quad (5)$$

where $W_2 \in \mathbb{R}^{F \times D}$ projects hyperedge features into a new node feature space of dimension F , and $a_2 \in \mathbb{R}^F$ is the node-level attention vector. The attention score is

$$\eta_{ik} = \text{LeakyReLU}([W_2 b_k \parallel W_1 x_i]) \odot H_{ik}, \quad (6)$$

where $\parallel$ denotes the concatenation operation that joins two vectors along their feature dimensions. This equation incorporates both the transformed hyperedge features and the original node features, modulated by the incidence weight H_{ik} . This formulation ensures that hyperedges with stronger structural affinity to node v_i exert greater influence on its updated representation, thereby capturing node-specific high-order structural context.

To integrate the structural information arising from the current topology and the recent evolution window, the final output of STGSA at timestamp t is expressed as

$$Z^{SG,t} = \tilde{X}^{cu,t} + \lambda_1 \tilde{X}^{fu,t}, \quad (7)$$

where $\tilde{X}^{cu,t}$ and $\tilde{X}^{fu,t}$ denote the node-level embeddings computed from $H^{cu,t}$ and $H^{fu,t}$, respectively, and λ_1 is a scaling coefficient that controls the contribution of recent structural changes. Concatenating the outputs across all timestamps yields $Z^{SG} = \{Z^{SG,1}, Z^{SG,2}, \dots, Z^{SG,T}\} \in \mathbb{R}^{N \times T \times F}$. This representation simultaneously captures global structural context and short-term structural evolution, and serves as the structural input for the subsequent long-term attention fusion module, thereby enhancing the model's ability to characterize dynamic graph evolution.

4.2. Local structure attention

We develop the Local Structure Attention (LSA) module to capture fine-grained topological interactions in the snapshot G^t of the graph at each timestamp t . This module operates on the node feature matrix $X^t = [x_1, x_2, \dots, x_N] \in \mathbb{R}^{N \times D}$ and the adjacency matrix A^t , and adaptively focuses on the importance weights of each node $v \in V^t$ and its neighborhood N_v via an attention mechanism. This enables the model to capture fine-grained local structural interactions between nodes. Finally, the embedding $z_v \in \mathbb{R}^F$ is integrated into the matrix $Z^{LS,t} = [z_1, z_2, \dots, z_N] \in \mathbb{R}^{N \times F}$. The representation of target node v is updated by aggregating transformed features from its immediate neighborhood N_v . Formally, the updated hidden representation h_v is formulated as a weighted sum of projected neighbor attributes:

$$h_v = \text{ELU} \left(\sum_{u \in N_v} \alpha_{uv} W_s x_u \right), \quad (8)$$

where $W_s \in \mathbb{R}^{F \times D}$ denotes the learnable weight matrix for linear transformation, and ELU denotes the ELU activation function. The attention score α_{uv} serves as a dynamic scalar that quantifies the relative influence of neighbor $u \in N_v$ on the representation of node v . Utilizing a learnable vector $a \in \mathbb{R}^{2F}$, the attention score α_{uv} is computed as follows:

$$\alpha_{uv} = \frac{\exp(\sigma(A_{uv} a^\top [W_s x_u \parallel W_s x_v]))}{\sum_{w \in N_v} \exp(\sigma(A_{uw} a^\top [W_s x_w \parallel W_s x_v]))}. \quad (9)$$

Here, $\parallel$ denotes the vector concatenation operation, and $\sigma(\cdot)$ denotes the LeakyReLU activation function. We leverage the sparse adjacency matrix

entry A_{uv} to efficiently implement masked attention to neighbors, as α_{uv} is zero for all unlinked nodes in G^t .

Building on this, we extend the attention paradigm to a multi-head approach. Multiple attention heads are designed within the LSA layer, each head learning a distinct set of projection parameters to capture specific structural patterns. The final embedding of node v is obtained by concatenating the outputs of these multi-head attention heads:

$$z_v = \text{Concat}(h_v^1, h_v^2, \dots, h_v^H), \quad \forall v \in V^t. \quad (10)$$

Here, H denotes the number of attention heads, and z_v represents the output embedding of node v after multi-head attention. Notably, the LSA layer is applied independently to each graph snapshot, while all attention parameters are shared across the temporal dimension T .

Upon completing the localized attention computation for each individual snapshot, the resulting structure-aware matrices $Z^{LS,t}$ are chronologically integrated to form a coherent spatiotemporal representation. Formally, the local structural matrices corresponding to all T timestamps are concatenated into a unified third-order tensor $Z^{LS} = [Z^{LS,1}, Z^{LS,2}, \dots, Z^{LS,T}] \in \mathbb{R}^{N \times T \times F}$, where N denotes the number of nodes, T denotes the total number of timestamps, and F denotes the dimension of node embedding. This tensor provides a unified local structure-aware signal, facilitating integration with the outputs from the STGSA and TFA modules in subsequent layers, thereby enabling the model to reconcile multi-scale dependencies within a consistent analytical framework.

4.3. Temporal feature attention

We propose the Temporal Feature Attention (TFA) module to comprehensively capture temporal dependencies while reflecting the evolving structural dynamics of nodes in dynamic graphs. This module models the evolutionary trajectories of node features across the temporal domain, thereby capturing fine-grained temporal dependencies. Through this mechanism, TFA captures time-dependent variations in node attributes, with particular emphasis on recent and causally preceding feature states.

Let v be a node in the dynamic graph. We define its temporal feature trajectory as $\{x_v^1, x_v^2, \dots, x_v^T\}$, where each feature vector $x_v^t \in \mathbb{R}^D$ describes the state of node v at timestamp t . For computational convenience, these vectors are arranged in chronological order to form a temporal feature matrix $X_v = [x_v^1, x_v^2, \dots, x_v^T] \in \mathbb{R}^{T \times D}$, which serves as the input to the TFA module. The output is an updated feature matrix $Z_v \in \mathbb{R}^{T \times F}$, where D and F denote the input and output feature dimensions, respectively, and T denotes the total number of time steps.

The TFA module employs a scaled dot-product attention mechanism (Vaswani et al., 2017) to model node representations across different timestamps. Specifically, the input matrix X_v is linearly projected into query, key, and value spaces via learnable transformation matrices $W_Q, W_K, W_V \in \mathbb{R}^{D \times F}$. To preserve the temporal order of information, we introduce a causal mask matrix, defined as follows:

$$M_{ij} = \begin{cases} 0, & i \leq j, \\ -\infty, & \text{otherwise,} \end{cases} \quad (11)$$

when $M_{ij} = -\infty$, the corresponding attention score becomes negligible after the softmax operation, thus eliminating the influence of future information and preserving autoregressive properties. By incorporating this directional masking strategy into the attention mechanism, TFA adaptively highlights significant and time-sensitive changes in node features while maintaining temporal causality. The update calculation of

node feature Z_v is thus formulated as:

$$\begin{aligned} Z_v &= \beta_v(X_v W_V), \\ \beta_v^{ij} &= \frac{\exp(e_v^{ij})}{\sum_{k=1}^T \exp(e_v^{ik})}, \\ e_v^{ij} &= \frac{(X_v W_Q)(X_v W_K)^T}{\sqrt{F}} + M_{ij}, \end{aligned} \quad (12)$$

where e_v^{ij} denotes the attention score from timestamp i to j , β_v^{ij} is the attention weight, and $M \in \mathbb{R}^{T \times T}$ is a masking matrix designed to enforce temporal causality.

Finally, for each graph snapshot $G' \in \mathcal{G}$, the learned node representations are organized into a matrix $Z' = [z'_1, z'_2, \dots, z'_N] \in \mathbb{R}^{N \times F}$, where z'_v represents the feature of node v at timestamp t . The complete spatio-temporal representations of all nodes over time, produced by the TFA module, are denoted as $Z^{TF} = [Z^{TF,1}, Z^{TF,2}, \dots, Z^{TF,T}] \in \mathbb{R}^{N \times T \times F}$, where N and T represent the total number of nodes and timestamps, and F denotes the node embedding dimension, respectively.

4.4. Long-term attention fusion

The Long-Term Attention Fusion (LTAF) module is designed to capture long-range temporal dependencies and the progressive evolution of structural patterns in dynamic graphs. Since node representations generated at each timestamp encode complementary perspectives—including local spatial context, global structural features, and temporal dynamics—the LTAF integrates these heterogeneous representations into a unified temporal feature matrix. Specifically, for each time step t , the fused representation is defined as

$$Z^{LT,t} = Z^{LS,t} + \lambda_2 Z^{SG,t} + \lambda_3 Z^{TF,t}, \quad (13)$$

where $Z^{LS,t} \in \mathbb{R}^{LS}$ denotes the node representations from the LSA module capturing local spatiotemporal interactions, $Z^{SG,t} \in \mathbb{R}^{SG}$ corresponds to the outputs of the STGSA module capturing global structural context, and $Z^{TF,t} \in \mathbb{R}^{TF}$ represents the TFA outputs encoding fine-grained temporal patterns. The hyperparameters $\lambda_2, \lambda_3 \in \mathbb{R}$ adjust the relative importance of global structural and temporal features in the fused embedding.

To explicitly encode temporal order, we add sinusoidal positional encodings $pos^t \in \mathbb{R}^F$ to each node's fused feature $z'_v \in \mathbb{R}^F$, yielding

$$\tilde{z}'_v = z'_v + pos^t, \quad (14)$$

where $v \in \{1, 2, \dots, N\}$ indexes nodes and F denotes the feature dimension. Aggregating all node sequences over T timestamps yields the tensor $\tilde{Z} \in \mathbb{R}^{N \times T \times F}$, where $N = |V|$ is the number of nodes.

In the temporal feature modeling stage of the Long-Term Attention Fusion module, each node v is represented by a temporal feature sequence constructed by integrating its representations across all timestamps. Formally, we denote this sequence as $\{\tilde{z}'_v, \tilde{z}'_v, \dots, \tilde{z}'_v\}$. To facilitate computations, these vectors are ordered chronologically to construct a temporal feature matrix $\tilde{Z}_v = [\tilde{z}'_v, \tilde{z}'_v, \dots, \tilde{z}'_v] \in \mathbb{R}^{T \times F}$. This temporal matrix encapsulates the evolution trajectory of node v 's attributes and serves as the input to the multi-head attention mechanism.

To make the temporal sequence compatible with multi-head attention computation, we introduce learnable projection matrices that map $\tilde{Z}_v$ into a latent space of dimension $F = Hd$, where H is the number of attention heads and d_k is the feature dimension per head. These linear transformations generate the query, key, and value matrices:

$$Q = \tilde{Z}_v \tilde{W}_Q, \quad K = \tilde{Z}_v \tilde{W}_K, \quad V = \tilde{Z}_v \tilde{W}_V, \quad (15)$$

where $\tilde{W}_Q, \tilde{W}_K, \tilde{W}_V \in \mathbb{R}^{F \times F}$ are learnable parameters shared across all nodes.

We then split the resulting matrices Q , K , and V into H parallel subspaces, forming head-specific projections $Q_h, K_h, V_h \in \mathbb{R}^{T \times d}$. For each head $h \in \{1, 2, \dots, H\}$, temporal dependencies are computed via scaled

dot-product attention and the causal mask matrix (Eq. (11)). The attention scores and output of head h are computed as

$$\theta_h^{ij} = \frac{\exp\left(\frac{(Q_h K_h^T)_{ij}}{\sqrt{d_k}} + M_{ij}\right)}{\sum_{k=1}^T \exp\left(\frac{(Q_h K_h^T)_{ik}}{\sqrt{d_k}} + M_{ik}\right)}, \quad (16)$$

$$Z_h = \theta_h V_h,$$

where θ_h^{ij} denotes the normalized attention weight from timestamp i to j under head h , and $Z_h \in \mathbb{R}^{T \times d_k}$ is the resulting weighted representation capturing head-specific temporal dependencies.

Finally, we concatenate the outputs from all attention heads to form the long-term temporal embedding of node v at timestamp t :

$$o_v^t = \text{Concat}(Z^{t,1}, Z^{t,2}, \dots, Z^{t,H}), \quad (17)$$

and the representation for all nodes at timestamp t is denoted as $O^t = [o_1^t, o_2^t, \dots, o_N^t] \in \mathbb{R}^{N \times F}$. The entire temporal evolution sequence is represented as $\mathcal{O} = \{O^1, O^2, \dots, O^T\}$. The resulting embedding thus captures both local and global temporal dependencies while preserving structural signals encoded in earlier fusion modules, providing a robust foundation for downstream tasks such as temporal link prediction and evolving node classification.

4.5. Model learning

MADGN optimizes node representations by jointly preserving connective proximity loss and structural similarity relationships, which together govern the dynamic behavior of evolving graph systems. To model the time-varying interactions among nodes, we introduce a connective proximity loss, formulated as:

$$\begin{aligned} \mathcal{L}_c = \sum_{t=1}^T \sum_{u=1}^N \left(\sum_{v \in \mathcal{N}_u^t} -\log \sigma(\langle o_u^t, o_v^t \rangle) \right. \\ \left. - \beta \sum_{v' \in \mathcal{P}_u^t} \log(1 - \sigma(\langle o_u^t, o_{v'}^t \rangle)) \right), \end{aligned} \quad (18)$$

where $\mathcal{N}_u^t$ denotes nodes that co-occur with u through random walks at time t , $\mathcal{P}_u^t$ denotes the negative sample set, $\sigma(\cdot)$ denotes the sigmoid function, and $\langle \cdot, \cdot \rangle$ denotes the inner product that quantifies pairwise affinity. Here, β denotes the negative sampling ratio. This adjustable hyperparameter controls the relative contribution of negative samples during optimization. Random walks approximate multi-hop structural context within each snapshot, allowing the model to preserve higher-order connective proximity. Through its contrastive formulation, the loss increases the similarity of temporally co-occurring nodes while suppressing spurious correlations introduced by negative samples, yielding embeddings that faithfully reflect the evolving interaction patterns of the graph.

In addition to connective proximity, dynamic graphs often exhibit persistent structural regularities, such as nodes that occupy comparable functional roles despite fluctuations in their immediate neighborhoods. To preserve such structural semantics, we impose a cosine-similarity-based regularization loss:

$$\mathcal{L}_s = \sum_{t=1}^T \sum_{u=1}^N \sum_{v \in S(u)} \left(1 - \frac{o_u^t \cdot o_v^t}{\|o_u^t\|_2 \|o_v^t\|_2} \right), \quad (19)$$

where $S(u)$ represents the set of nodes structurally similar to u . Cosine similarity focuses on the directional alignment between embeddings, making it suitable for capturing structural equivalence independent of scale. This regularization constrains nodes with analogous roles to remain close in the embedding space, reducing undesirable variation in structural-role representations across time.

The overall optimization objective integrates both temporal and structural constraints:

$$\mathcal{L} = \mathcal{L}_c + \zeta \mathcal{L}_s, \quad (20)$$

Table 1

Theoretical time complexity of representative dynamic graph models.

Model	Time Complexity
HTGN (Yang et al., 2021)	$O(NWd' + Ndd' + E d')$
CTGCN (Liu et al., 2022)	$O(N(d + d')k' + lk E)$
SpikeNet (Li et al., 2023)	$O(Nd^2S^k)$
DGCN (Gao et al., 2023)	$O(E c_{\text{avg}} \log c_{\text{avg}})$
MTSN (Liu et al., 2021)	$O(Ndd' + Ld E)$
RTGCN (Du et al., 2024)	$O(E + E Nd')$
MADGN (Ours)	$O(E + \mathcal{E} + N)$

Notations: N , $|E|$, and T denote the numbers of nodes, edges, and snapshots, respectively; d , d' , and F denote feature dimensions. The formulas are expressed using unified notation and dominant computational terms.

where ζ denotes a hyperparameter that balances the contribution of structural similarity relative to connective proximity. By jointly optimizing these complementary objectives, MADGN learns dynamic representations that reflect both transient interaction patterns and persistent structural information. This enables the model to maintain robustness under rapid topological changes while capturing fine-grained temporal dependencies essential for downstream prediction tasks.

4.6. Complexity analysis

This section analyzes the computational complexity of the proposed MADGN framework. Consistent with common practice in dynamic graph representation learning, we adopt an operator-level complexity analysis, where feature dimensions, the number of attention heads, and internal neural transformations are treated as constants. Consequently, the analysis focuses on the structural and temporal scales of the dynamic graph. Let N denote the number of nodes, $|E|$ the number of edges in a single snapshot, T the number of temporal snapshots, and $|\mathcal{E}|$ the number of node-hyperedge incidences in the short-term structural similarity hypergraph.

The Short-Term Global Structure Attention module constructs a structural similarity hypergraph and conducts dual-level aggregation at both the node and hyperedge levels, incurring a computational cost linear in the number of node-hyperedge incidences in the hypergraph $O(|\mathcal{E}| + N)$. The Local Structure Attention module adopts parallel neighborhood-level message passing, whose computational cost is proportional to the number of edges in the snapshot graph $O(|E|)$. Under the single-snapshot setting, Temporal Feature Attention degenerates into a linear transformation of node features, incurring a computational overhead of $O(N)$. Furthermore, MADGN employs a Long-Term Attention Fusion mechanism to integrate historical node representations across temporal snapshots. For each node, this module performs full attention over its representations across T time steps, resulting in a per-node cost of $O(T^2)$. Under the operator-level complexity analysis paradigm assumption, the overall computational cost of the temporal modeling component is therefore $O(T^2N)$.

Table 1 presents a comparison of the time complexity across different models. For prediction tasks performed on each snapshot, the overall time complexity is $O(T(|E| + |\mathcal{E}| + TN))$. Notably, the time complexity of our method scales linearly with the number of edges. Compared with existing dynamic graph learning methods that adopt recurrent parameter evolution or full temporal expansion, this renders MADGN suitable for large-scale graph tasks. Our subsequent experimental results further validate these findings.

Algorithm 1 summarizes the complete training procedure of MADGN. The time-aware structural-similarity hypergraphs are constructed once before optimization and remain fixed during model training. At each epoch, STGSA and LSA independently encode high-order and local structural information for individual snapshots, whereas TFA

models the causally ordered evolution of node attributes across timestamps. The resulting complementary representations are subsequently integrated and processed by LTAF to capture long-term dependencies. Finally, the connective-proximity and structural-similarity objectives are jointly optimized through backpropagation.

5. Experiments and results

5.1. Experimental setup

5.1.1. Datasets

DBLP-3/DBLP-5 are large-scale academic collaboration networks derived from the DBLP computer science bibliography. Specifically, **DBLP-3** focuses on a longitudinal subset of co-authorship interactions, while **DBLP-5** encompasses a broader range of research domains with distinct topological evolution patterns. These datasets are utilized to evaluate the model's ability to capture complex relational dynamics in academic ecosystems.

Alibaba is a real-world user behavior dataset from the Alibaba e-commerce platform. It contains fine-grained, time-stamped interaction records, providing a robust benchmark for evaluating dynamic node embedding in highly sparse and evolving industrial scenarios.

Epinions originates from a consumer review portal where users establish explicit trust and distrust relationships. This dataset facilitates the assessment of our model in preserving both social trust dynamics and rating-based structural transitions. Detailed statistics of the datasets are summarized in Table 2.

5.1.2. Evaluation metrics

We evaluated MADGN and all baseline models on dynamic link prediction, dynamic new link prediction, and node classification tasks.

For dynamic link prediction and dynamic new link prediction, we report AUC and F1 metrics. AUC measures a model's ability to rank positive edges (connected edges) higher than negative edges (unconnected edges) without relying on a specific decision threshold. The F1 evaluates the balance between precision and recall at a selected threshold. To prevent information leakage from the test set, the decision threshold used for the F1-score is determined based on performance on the validation set and subsequently applied to the test set. Positive edges are derived from links observed in the corresponding evaluation snapshot, while negative edges are sampled from pairs of unconnected nodes according to a standard protocol.

For the node classification task, we report accuracy (ACC) and multi-class AUC. ACC measures the proportion of test nodes correctly classified, whereas AUC evaluates the classifier's ranking capability across all classes within the task setting.

5.1.3. Baselines

To validate the effectiveness of MADGN, we compare it with 16 dynamic graph representation learning baselines, covering mainstream technical paradigms such as recurrent temporal modeling, temporal attention mechanisms, higher-order structure modeling, and event-driven temporal dynamic modeling. Table 3 systematically summarizes the design differences between MADGN and all baseline methods in each core modeling dimension, clearly presenting the differentiated modeling capabilities of each method.

DGCN (Gao et al., 2023) updates GCN parameters with LSTM to capture dynamic structural changes.

DynaAERNN (Goyal et al., 2020) learns temporal transitions using dense and recurrent neural layers.

EvolveGCN (Pareja et al., 2020) evolves GCN parameters over time through recurrent networks.

DySAT (Sankar et al., 2020) combines structural self-attention and temporal self-attention for dynamic node representation learning.

Algorithm 1: Training procedure of MADGN.

Input: Dynamic graph sequence $\mathcal{G} = \{G^1, \dots, G^T\}$; adjacency matrices $\mathcal{A} = \{A^1, \dots, A^T\}$; node attributes $\mathcal{X} = \{X^1, \dots, X^T\}$.
Hyperparameters: History window k ; temporal decay rate ω ; fusion coefficients $\lambda_1, \lambda_2, \lambda_3$; structural loss weight ζ ; maximum number of epochs I .
Output: Dynamic node embeddings $\mathcal{O} = \{O^1, \dots, O^T\} \in \mathbb{R}^{N \times T \times F}$.

```

// Stage I: Time-aware structural similarity hypergraph construction
1 for  $t \leftarrow 1$  to  $T$  do
2    $H^{cu,t} \leftarrow \text{BuildHypergraph}(A^t)$ ; ▷ Current structural similarity hypergraph
3   for  $\tau \leftarrow 1$  to  $\min(k, t-1)$  do
4      $H^{fu,t} \leftarrow H^{fu,t} + \exp(-\omega \Delta t_\tau) H^{cu,t-\tau}$ ; ▷ Fusion structural similarity hypergraph

// Stage II: Joint multi-scale representation learning
5 Initialize trainable parameters  $\Theta$  and the Adam optimizer;
6 for  $r \leftarrow 1$  to  $I$  do
  // Snapshot-level structural encoding
  7 for  $t \leftarrow 1$  to  $T$  do
    8    $\tilde{X}^{cu,t} \leftarrow \text{HyperAttn}(X^t, H^{cu,t}; \Theta)$ ; ▷ Current-hypergraph branch
    9    $\tilde{X}^{fu,t} \leftarrow \text{HyperAttn}(X^t, H^{fu,t}; \Theta)$ ; ▷ Temporally fused branch
    10   $Z^{SG,t} \leftarrow \tilde{X}^{cu,t} + \lambda_1 \tilde{X}^{fu,t}$ ; ▷ Short-term global representation
    11   $Z^{LS,t} \leftarrow \text{LSA}(A^t, X^t; \Theta)$ ; ▷ Local pairwise representation
    12   $Z^{TF} = \{Z^{TF,1}, \dots, Z^{TF,T}\} \leftarrow \text{TFA}(\mathcal{X}; \Theta)$ ; ▷ Temporal feature representation
    13  for  $t \leftarrow 1$  to  $T$  do
    14     $Z^{LT,t} \leftarrow Z^{LS,t} + \lambda_2 Z^{SG,t} + \lambda_3 Z^{TF,t}$ ; ▷ Fusion representation

    // Long-term temporal fusion
    15   $\mathcal{O} \leftarrow \text{LTAF}(\{Z^{LT,1}, \dots, Z^{LT,T}\}; \Theta)$ ; ▷ Long-term representation
    // Joint representation optimization
    16   $\mathcal{L} \leftarrow \mathcal{L}_c + \zeta \mathcal{L}_s$ ;
    17  Update  $\Theta$  using Adam;
18 return  $\mathcal{O}$ ;

```

Table 2
Statistics of the evaluated datasets.

Dataset	Epinions	Alibaba	DBLP-3	DBLP-5
Nodes	9398	5640	4257	6606
Edges	231,537	53,049	23,540	42,815
Classes	–	–	3	5
Features	44	–	100	100
Snapshots	9	11	10	10
Domain	Social	E-commerce	Academic	Academic

GCRN (Seo et al., 2018) integrates graph convolution with recurrent neural networks to jointly model spatial and temporal dependencies.

ROLAND (You et al., 2022) treats node embeddings at different GNN layers as hierarchical node states and recurrently updates them over time.

RTGCN (Du et al., 2024) introduces structural role-based hypergraphs to capture global structural role information.

CTGCN (Liu et al., 2022) preserves both local connective proximity and global structural similarity in dynamic graphs.

MTSN (Liu et al., 2021) uses motif-preserving encoders and temporal shift operations to model high-order structure and temporal dynamics in dynamic attributed networks.

SpikeNet (Li et al., 2023) employs spiking neural networks to capture temporal graph evolution.

GAEN (Shi et al., 2021) preserves node similarities derived from time-varying patterns and evolves model weights according to topological changes.

HTGN (Yang et al., 2021) maps temporal graphs into hyperbolic space and combines hyperbolic graph neural networks with recurrent temporal modeling.

5.1.4. Experimental settings and implementation details

Initial node features: In this experiment, most benchmark datasets used for temporal graph representation learning contain only topological information. For the Alibaba dataset, we employ an identity matrix as the node features, consistent with the approach in Du et al. (2024) and You et al. (2022). In contrast, the nodes in the DBLP3, DBLP5, and Epinions datasets have their own node features, and our experiments directly use their original features.

Dataset splitting: For the link prediction task, we adopt the same experimental setup as MTSN (Liu et al., 2021). For each prediction step $t \rightarrow t+1$, node embeddings are learned using snapshots up to t . Positive edges in G^{t+1} are divided into 20% training, 20% validation, and 60% test sets solely for training and evaluating the downstream logistic-regression classifier. No edge from G^{t+1} is used during representation learning. For the node classification task, 70% of nodes are used for training, an extra 20% for validation, and the last 10% for testing.

Parameter Settings: We adopt a unified downstream evaluation protocol. For the edge prediction task, we follow the settings in Liu et al. (2022): we first compute the Hadamard product of the node embeddings corresponding to the target edge to obtain edge features, and then train a logistic regression classifier based on these features to determine the existence of an edge between nodes. For the node classification task, a logistic regression classifier is trained directly on the learned node embeddings to predict node classes. This evaluation paradigm is standard in the field of dynamic graph representation learning; all methods utilize a unified node-level split for training, validation, and testing, ensuring fair comparison within the same downstream classification framework.

Regarding baseline methods, we prioritize reproducing results using the official source code released by the original authors; if the official code is unavailable or incompatible with our experimental environment, we re-implement the core architecture and training objective following the corresponding publications. Initial hyperparameter settings for each

Table 3

Comparison of explicit modeling components in representative dynamic graph methods.

Methods	Local topology	High-order structure	Dynamic attributes	Long-range temporal modeling	Unified fusion
DGCN (Gao et al., 2023)	Full	No	Partial	Partial	No
DynAERNN (Goyal et al., 2020)	Partial	No	Partial	Partial	No
EvolveGCN (Pareja et al., 2020)	Full	No	Partial	Partial	No
DySAT (Sankar et al., 2020)	Full	Partial	Partial	Full	Partial
GCRN (Seo et al., 2018)	Full	No	Partial	Partial	No
ROLAND (You et al., 2022)	Full	Partial	Partial	Full	Partial
RTGCN (Du et al., 2024)	Full	Full	Partial	Partial	Partial
CTGCN (Liu et al., 2022)	Full	Full	No	Partial	Partial
MTSN (Liu et al., 2021)	Full	Full	Full	Partial	Partial
SpikeNet (Li et al., 2023)	Partial	No	Partial	Full	No
GAEN (Shi et al., 2021)	Full	Partial	Partial	Partial	No
HTGN (Yang et al., 2021)	Full	Partial	Partial	Full	Partial
MADGN (ours)	Full	Full	Full	Full	Full

Notations: Full: explicitly modeled; Partial: indirectly or conditionally modeled; No: not explicitly modeled.

baseline method follow the recommendations in the original literature. The maximum number of training epochs is set to 200 for all models, and an early stopping strategy based on validation set performance is employed. All experiments are repeated using five different random seeds, and the average results are reported as the final performance metrics. Specifically: for random walk-based methods, we set the window size to 6, the number of negative samples to 8, the number of walks per node to 100, and the walk length to 10; for GNN-based baselines, the number of graph convolutional layers is set to 2 by default. For dynAE and dynAERNN, we retain the original settings: a batch size of 5, a look-back length of 1, and regularization parameters γ_1 and γ_2 both set to 10^{-6} .

For our proposed MADGN model, considering both validation set performance and parameter sensitivity analysis results, we set the hidden layer dimension to 128 and employ 8 attention heads for both the structural and temporal attention layers. The negative sampling ratio, temporal decay factor, and structural similarity loss weight are set to 10, 1, and 5, respectively; the default values selected according to validation-set performance under the unified experimental configuration. Unless otherwise specified, all datasets use the same default configuration; furthermore, we verified the parameter robustness of MADGN through extensive parameter sweeps.

5.2. Overall performance

We evaluate the effectiveness of MADGN on three downstream tasks: dynamic link prediction, dynamic new link prediction, and node classification. Tables 4–6 report the average results over five independent runs. The best results are highlighted in bold, and the second-best results are underlined. The relative improvement is computed with respect to the strongest competing baseline for each metric. In our proposed framework, the link prediction and node classification tasks share the same latent representation space, allowing the transferability of the learned representations to be evaluated across different downstream tasks.

5.2.1. Dynamic link prediction

As shown in Table 4, MADGN achieves the best AUC and F1-score on all four datasets. Compared with RTGCN, which is the strongest competing baseline in Table 4, MADGN obtains relative AUC improvements of 4.66%, 0.91%, 2.77%, and 0.53% on Alibaba, Epinions, DBLP-3, and DBLP-5, respectively. The corresponding F1 improvements are 2.20%, 3.65%, 2.66%, and 0.89%.

The performance gains exhibit different characteristics across datasets. On Alibaba, where informative raw node attributes are unavailable, MADGN improves the AUC from 0.902 to 0.944. This result indicates that jointly modeling local connectivity, high-order structural similarity, and temporal evolution provides effective predictive information even when node semantics are limited. On Epinions and DBLP-5, the AUC margins are relatively small because the strongest baseline already achieves high ranking performance. Nevertheless, MADGN improves the

F1 on Epinions from 0.960 to 0.995, suggesting that the learned representations also provide more reliable discrimination.

Among the compared methods, RTGCN consistently provides the strongest baseline performance, demonstrating the importance of explicit structural-role modeling. The further improvements achieved by MADGN indicate that high-order structural information is more effective when coordinated with local topology, temporal node attributes, and long-term temporal dependencies. The consistently small standard deviations of MADGN also demonstrate stable optimization across different random seeds.

5.2.2. Dynamic new link prediction

Table 5 presents the results for predicting links between node pairs without historical interactions. MADGN again achieves the best performance on every dataset and metric. Relative to the strongest baseline, it improves AUC by 6.17%, 5.17%, 4.11%, and 2.70% on Alibaba, Epinions, DBLP-3, and DBLP-5, respectively, while the corresponding F1 improvements are 6.34%, 10.81%, 4.18%, and 2.72%.

The gains are generally larger than those observed in dynamic link prediction, particularly on Alibaba and Epinions. The largest improvement occurs on Epinions, where MADGN increases the F1-score from 0.897 to 0.994. Since historical adjacency information is unavailable for the evaluated node pairs, successful new-link prediction requires transferable signals beyond previously observed edges. The results therefore provide evidence that the structural similarity hypergraph and temporal attention mechanisms capture informative relationships among structurally related but previously unconnected nodes.

5.2.3. Node classification

To evaluate the versatility and transferability of the learned embeddings, we additionally carry out node classification experiments. Table 6 reports the node classification results using a unified logistic-regression evaluator. MADGN achieves the highest ACC and AUC on both DBLP-3 and DBLP-5. On DBLP-3, MADGN obtains an ACC of 0.774 and an AUC of 0.901, outperforming RTGCN by 14.5% and 7.9%, respectively. On DBLP-5, MADGN achieves an ACC of 0.707 and an AUC of 0.898, corresponding to relative improvements of 2.5% and 2.6%.

The particularly large ACC improvement on DBLP-3 indicates that the learned embeddings provide clearer class-discriminative information rather than merely improving pairwise link scores. On DBLP-5, where the five-class setting requires more fine-grained semantic separation, the improvement margin is smaller but remains consistent for both ACC and AUC. These results show that the advantages of MADGN extend beyond structural edge inference and transfer to node-level semantic prediction.

Overall, MADGN achieves superior performance across all datasets, downstream tasks, and evaluation metrics. The significant gains observed in the dynamic new link prediction task highlight the value of

Table 4

Dynamic link prediction results on four datasets. Higher values are better.

Methods	Alibaba		Epinions		DBLP-3		DBLP-5	
	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$
GCRN	0.711 $\pm$ 0.004	0.647 $\pm$ 0.004	0.924 $\pm$ 0.002	0.871 $\pm$ 0.001	0.799 $\pm$ 0.011	0.726 $\pm$ 0.008	0.832 $\pm$ 0.005	0.755 $\pm$ 0.006
DySAT	0.815	0.77	0.91	0.847	0.687 $\pm$ 0.009	0.649 $\pm$ 0.010	0.701 $\pm$ 0.021	0.682 $\pm$ 0.013
dynAE	0.645	0.615	0.556	0.663	0.742 $\pm$ 0.004	0.713 $\pm$ 0.005	0.796 $\pm$ 0.006	0.732 $\pm$ 0.007
dynAERNN	0.584	0.564	0.532	0.644	0.638 $\pm$ 0.016	0.624 $\pm$ 0.014	0.624 $\pm$ 0.024	0.671 $\pm$ 0.037
EvolveGCN	0.876 $\pm$ 0.007	0.827 $\pm$ 0.009	0.647 $\pm$ 0.011	0.637 $\pm$ 0.013	0.825 $\pm$ 0.009	0.752 $\pm$ 0.013	0.775 $\pm$ 0.009	0.714 $\pm$ 0.007
CTGCN-C	0.821 $\pm$ 0.002	0.730 $\pm$ 0.001	0.968 $\pm$ 0.003	0.905 $\pm$ 0.004	0.777 $\pm$ 0.003	0.716 $\pm$ 0.006	0.784 $\pm$ 0.006	0.712 $\pm$ 0.003
CTGCN-S	0.819 $\pm$ 0.004	0.745 $\pm$ 0.003	0.977 $\pm$ 0.000	0.922 $\pm$ 0.001	0.723 $\pm$ 0.007	0.701 $\pm$ 0.005	0.780 $\pm$ 0.003	0.733 $\pm$ 0.003
GAEN	0.806 $\pm$ 0.006	0.733 $\pm$ 0.004	0.747 $\pm$ 0.008	0.713 $\pm$ 0.010	0.783 $\pm$ 0.011	0.721 $\pm$ 0.012	0.846 $\pm$ 0.013	0.771 $\pm$ 0.009
HTGN	0.884 $\pm$ 0.007	0.832 $\pm$ 0.008	0.926 $\pm$ 0.002	0.867 $\pm$ 0.003	0.751 $\pm$ 0.019	0.687 $\pm$ 0.010	0.851 $\pm$ 0.012	0.777 $\pm$ 0.011
TSN	0.871	0.832	0.919	0.862	0.873 $\pm$ 0.017	0.773 $\pm$ 0.012	0.863 $\pm$ 0.028	0.764 $\pm$ 0.011
MTSN	0.886	0.842	0.931	0.879	0.881 $\pm$ 0.012	0.782 $\pm$ 0.009	0.874 $\pm$ 0.016	0.769 $\pm$ 0.008
DGCN	0.810 $\pm$ 0.005	0.739 $\pm$ 0.004	0.734 $\pm$ 0.008	0.660 $\pm$ 0.006	0.838 $\pm$ 0.007	0.791 $\pm$ 0.005	0.804 $\pm$ 0.009	0.742 $\pm$ 0.007
ROLAND-s	0.838 $\pm$ 0.020	0.786 $\pm$ 0.023	0.926 $\pm$ 0.025	0.860 $\pm$ 0.021	0.896 $\pm$ 0.021	0.853 $\pm$ 0.011	0.890 $\pm$ 0.011	0.837 $\pm$ 0.015
ROLAND-l	0.888 $\pm$ 0.009	0.824 $\pm$ 0.006	0.938 $\pm$ 0.009	0.881 $\pm$ 0.004	0.904 $\pm$ 0.015	0.864 $\pm$ 0.012	0.895 $\pm$ 0.005	0.843 $\pm$ 0.006
SpikeNet	0.839 $\pm$ 0.005	0.768 $\pm$ 0.003	0.862 $\pm$ 0.011	0.803 $\pm$ 0.014	0.826 $\pm$ 0.011	0.781 $\pm$ 0.009	0.816 $\pm$ 0.013	0.769 $\pm$ 0.011
RTGCN	0.902 $\pm$ 0.002	0.864 $\pm$ 0.001	0.989 $\pm$ 0.001	0.960 $\pm$ 0.001	0.937 $\pm$ 0.001	0.901 $\pm$ 0.001	0.946 $\pm$ 0.003	0.898 $\pm$ 0.001
MADGN (ours)	0.944 $\pm$ 0.001	0.883 $\pm$ 0.001	0.998 $\pm$ 0.001	0.995 $\pm$ 0.001	0.963 $\pm$ 0.001	0.925 $\pm$ 0.001	0.951 $\pm$ 0.001	0.906 $\pm$ 0.001
Improve.(%)	4.66%	2.20%	0.91%	3.65%	2.77%	2.66%	0.53%	0.89%

Table 5

Dynamic new-link prediction results on four datasets. Higher values are better.

Methods	Alibaba		Epinions		DBLP-3		DBLP-5	
	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$
GCRN	0.592 $\pm$ 0.006	0.565 $\pm$ 0.004	0.763 $\pm$ 0.008	0.694 $\pm$ 0.006	0.765 $\pm$ 0.005	0.698 $\pm$ 0.004	0.791 $\pm$ 0.007	0.719 $\pm$ 0.005
DySAT	0.809 $\pm$ 0.007	0.761 $\pm$ 0.006	0.896 $\pm$ 0.009	0.829 $\pm$ 0.007	0.654 $\pm$ 0.004	0.625 $\pm$ 0.005	0.651 $\pm$ 0.006	0.641 $\pm$ 0.005
dynAE	0.632 $\pm$ 0.023	0.601 $\pm$ 0.011	0.531 $\pm$ 0.006	0.634 $\pm$ 0.003	0.723 $\pm$ 0.011	0.657 $\pm$ 0.004	0.768 $\pm$ 0.013	0.670 $\pm$ 0.005
dynAERNN	0.535 $\pm$ 0.009	0.543 $\pm$ 0.019	0.519 $\pm$ 0.002	0.627 $\pm$ 0.004	0.570 $\pm$ 0.013	0.608 $\pm$ 0.007	0.601 $\pm$ 0.012	0.661 $\pm$ 0.002
EvolveGCN	0.690 $\pm$ 0.012	0.666 $\pm$ 0.010	0.550 $\pm$ 0.004	0.610 $\pm$ 0.004	0.741 $\pm$ 0.014	0.700 $\pm$ 0.012	0.759 $\pm$ 0.005	0.702 $\pm$ 0.012
CTGCN-C	0.766 $\pm$ 0.006	0.692 $\pm$ 0.004	0.836 $\pm$ 0.004	0.722 $\pm$ 0.005	0.745 $\pm$ 0.006	0.685 $\pm$ 0.005	0.760 $\pm$ 0.006	0.691 $\pm$ 0.005
CTGCN-S	0.777 $\pm$ 0.003	0.711 $\pm$ 0.009	0.831 $\pm$ 0.005	0.729 $\pm$ 0.008	0.715 $\pm$ 0.002	0.654 $\pm$ 0.003	0.763 $\pm$ 0.003	0.711 $\pm$ 0.002
GAEN	0.737 $\pm$ 0.004	0.704 $\pm$ 0.003	0.611 $\pm$ 0.005	0.601 $\pm$ 0.004	0.768 $\pm$ 0.006	0.704 $\pm$ 0.004	0.819 $\pm$ 0.004	0.745 $\pm$ 0.005
HTGN	0.835 $\pm$ 0.010	0.797 $\pm$ 0.008	0.880 $\pm$ 0.001	0.823 $\pm$ 0.004	0.700 $\pm$ 0.018	0.649 $\pm$ 0.013	0.800 $\pm$ 0.015	0.738 $\pm$ 0.018
TSN	0.778 $\pm$ 0.008	0.734 $\pm$ 0.009	0.812 $\pm$ 0.003	0.765 $\pm$ 0.005	0.690 $\pm$ 0.004	0.682 $\pm$ 0.007	0.753 $\pm$ 0.004	0.716 $\pm$ 0.002
MTSN	0.801 $\pm$ 0.006	0.776 $\pm$ 0.005	0.865 $\pm$ 0.004	0.812 $\pm$ 0.004	0.824 $\pm$ 0.004	0.743 $\pm$ 0.005	0.812 $\pm$ 0.005	0.738 $\pm$ 0.006
DGCN	0.803 $\pm$ 0.003	0.726 $\pm$ 0.004	0.707 $\pm$ 0.004	0.619 $\pm$ 0.003	0.832 $\pm$ 0.004	0.773 $\pm$ 0.004	0.798 $\pm$ 0.002	0.727 $\pm$ 0.003
ROLAND-s	0.814 $\pm$ 0.011	0.766 $\pm$ 0.012	0.910 $\pm$ 0.009	0.846 $\pm$ 0.005	0.859 $\pm$ 0.007	0.836 $\pm$ 0.009	0.879 $\pm$ 0.009	0.822 $\pm$ 0.010
ROLAND-l	0.843 $\pm$ 0.005	0.812 $\pm$ 0.004	0.917 $\pm$ 0.003	0.850 $\pm$ 0.005	0.871 $\pm$ 0.004	0.842 $\pm$ 0.003	0.885 $\pm$ 0.003	0.822 $\pm$ 0.006
SpikeNet	0.822 $\pm$ 0.002	0.753 $\pm$ 0.003	0.835 $\pm$ 0.004	0.779 $\pm$ 0.003	0.777 $\pm$ 0.010	0.728 $\pm$ 0.008	0.797 $\pm$ 0.003	0.745 $\pm$ 0.008
RTGCN	0.891 $\pm$ 0.002	0.836 $\pm$ 0.003	0.948 $\pm$ 0.002	0.897 $\pm$ 0.004	0.924 $\pm$ 0.003	0.885 $\pm$ 0.005	0.928 $\pm$ 0.001	0.884 $\pm$ 0.002
MADGN (ours)	0.946 $\pm$ 0.001	0.889 $\pm$ 0.001	0.997 $\pm$ 0.001	0.994 $\pm$ 0.001	0.962 $\pm$ 0.001	0.922 $\pm$ 0.001	0.953 $\pm$ 0.001	0.908 $\pm$ 0.001
Improve.(%)	6.17%	6.34%	5.17%	10.81%	4.11%	4.18%	2.70%	2.72%

Table 6

Node classification results on DBLP-3 and DBLP-5. Higher values are better.

Methods	DBLP-3		DBLP-5	
	ACC $\uparrow$	AUC $\uparrow$	ACC $\uparrow$	AUC $\uparrow$
GCRN	0.562 $\pm$ 0.008	0.738 $\pm$ 0.005	0.626 $\pm$ 0.005	0.836 $\pm$ 0.003
DySAT	0.549 $\pm$ 0.016	0.729 $\pm$ 0.018	0.520 $\pm$ 0.008	0.765 $\pm$ 0.016
dynAE	0.389 $\pm$ 0.005	0.612 $\pm$ 0.009	0.347 $\pm$ 0.003	0.623 $\pm$ 0.005
dynAERNN	0.321 $\pm$ 0.024	0.482 $\pm$ 0.017	0.272 $\pm$ 0.012	0.501 $\pm$ 0.015
EvolveGCN	0.547 $\pm$ 0.011	0.765 $\pm$ 0.006	0.638 $\pm$ 0.011	0.847 $\pm$ 0.012
CTGCN-C	0.630 $\pm$ 0.003	0.781 $\pm$ 0.007	0.665 $\pm$ 0.010	0.857 $\pm$ 0.005
CTGCN-S	0.591 $\pm$ 0.011	0.756 $\pm$ 0.004	0.650 $\pm$ 0.005	0.854 $\pm$ 0.002
GAEN	0.557 $\pm$ 0.007	0.693 $\pm$ 0.008	0.524 $\pm$ 0.009	0.776 $\pm$ 0.010
HTGN	0.556 $\pm$ 0.003	0.712 $\pm$ 0.004	0.538 $\pm$ 0.005	0.652 $\pm$ 0.009
TSN	0.444 $\pm$ 0.006	0.519 $\pm$ 0.007	0.276 $\pm$ 0.009	0.521 $\pm$ 0.019
MTSN	0.604 $\pm$ 0.004	0.669 $\pm$ 0.005	0.501 $\pm$ 0.007	0.754 $\pm$ 0.010
DGCN	0.613 $\pm$ 0.004	0.754 $\pm$ 0.005	0.668 $\pm$ 0.006	0.859 $\pm$ 0.005
ROLAND-s	0.575 $\pm$ 0.008	0.745 $\pm$ 0.008	0.579 $\pm$ 0.010	0.703 $\pm$ 0.011
ROLAND-l	0.625 $\pm$ 0.009	0.797 $\pm$ 0.007	0.634 $\pm$ 0.003	0.768 $\pm$ 0.005
SpikeNet	0.519 $\pm$ 0.007	0.656 $\pm$ 0.001	0.548 $\pm$ 0.011	0.703 $\pm$ 0.007
RTGCN	0.676 $\pm$ 0.004	0.835 $\pm$ 0.003	0.690 $\pm$ 0.003	0.875 $\pm$ 0.002
MADGN (ours)	0.774 $\pm$ 0.001	0.901 $\pm$ 0.001	0.707 $\pm$ 0.001	0.898 $\pm$ 0.001
Improve.(%)	14.5%	7.9%	2.5%	2.6%

modeling high-order structural similarity in the absence of direct interaction history, while the node classification results demonstrate that the learned representations also preserve transferable semantic information. Collectively, these findings validate the effectiveness of synergistically integrating local topology, global high-order structure, dynamic node attributes, and multi-scale temporal dependencies within a unified representation framework.

5.3. Ablation study

To systematically examine the role of each individual component within the MADGN framework, we performed rigorous ablation analyses. Specifically, by gradually isolating or deactivating specific attention modules, we quantified their respective impacts on the overall model performance. This ablation study comprises two main parts: the Component Ablation and the Effect of Hypergraph Construction Method.

5.3.1. Component ablation

w/o LSA: In this variant, the Local Structure Attention module was removed, preventing the model from explicitly modeling the structural relationships of nodes within their local neighborhoods. This setup is designed to assess the importance of local structural information in dynamic graph representation learning.

Table 7
Component ablation results for MADGN.

Module	DBLP-5		Alibaba		DBLP-3	
	F1 $\uparrow$	ACC $\uparrow$	AUC $\uparrow$	F1 $\uparrow$	AUC $\uparrow$	F1 $\uparrow$
w/o LSA	0.903	0.913	0.934	0.872	0.871	0.902
w/o TFA	0.912	0.932	0.629	0.585	0.961	0.917
w/o LTAF	0.894	0.893	0.612	0.580	0.953	0.908
w/o STGSA	0.920	0.945	0.937	0.875	0.962	0.924
w/o SGSA	0.919	0.940	0.940	0.876	0.953	0.908
MADGN	0.921	0.947	0.944	0.883	0.964	0.925

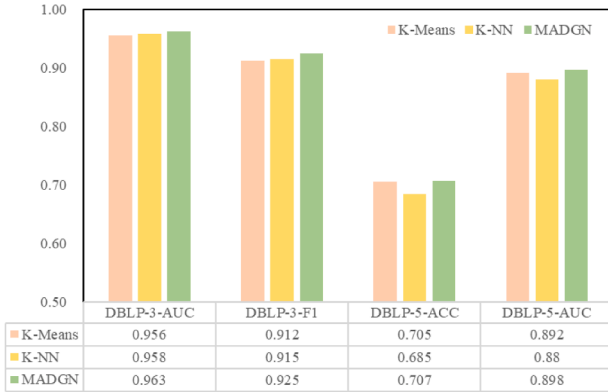


Fig. 3. Comparison of hypergraph construction strategies on DBLP-3 link prediction and DBLP-5 node classification.

w/o TFA: This variant involves the removal of the Temporal Feature Attention module, thereby disabling the model’s ability to attend differentially to temporal features. This experiment verifies the model’s reliance on this module for capturing critical temporal change patterns.

w/o LTAF: With the Long-Term Attention Fusion module ablated, the model can no longer leverage long-range temporal dependencies and must rely solely on short-term information or other structural features for inference. This variant aims to analyze the impact of long-term temporal dependencies on performance.

w/o STGSA: Removing the Short-Term Global Structure Attention module deprives the model of the capacity to capture global structural patterns at the current time step. This controlled experiment helps elucidate the contribution of short-term global structural information to learning stable representations.

w/o SGSA: This variant goes a step further by removing the step-wise global structural attention sub-module within the STGSA. Consequently, the model cannot explicitly model the influence of the previous time step’s global structure on the current one, allowing us to evaluate the importance of step-wise structural dependencies.

As shown in Table 7, the complete MADGN model achieves the best performance across all datasets and evaluation metrics, which supports the effectiveness of its multi-scale attention architecture. The overall trend from the ablation studies indicates that while removing any key component typically leads to performance degradation, the extent of this decline varies significantly depending on the specific module and dataset, revealing the specialization and complementarity of each module’s function. Specifically, the impact of module removal is most pronounced on the Alibaba dataset. When the Temporal Feature Attention (TFA) or Long-Term Attention Fusion (LTAF) modules are ablated, the model exhibits the most severe performance drop in both AUC and F1 scores (e.g., AUC drops from 0.944 to 0.629 for W/O TFA, and F1 drops from 0.883 to 0.580 for W/O LTAF). This finding indicates that temporal modeling is particularly important for the evaluated Alibaba dataset, and the model heavily relies on these modules to capture essential temporal evolution patterns. In contrast, on the DBLP-5 and DBLP-3 datasets, the performance gap between most ablated variants and the

complete model is relatively small. Particularly on DBLP-3, all variants maintain F1 scores above 0.902 (compared to 0.925 for the complete model). This indicates that MADGN possesses a robust foundational representation capacity, enabling it to maintain relatively stable performance on graph data with simpler structures or less volatile dynamics, even in the absence of some advanced modules. Furthermore, the removal of structural modules (e.g., LSA, STGSA, SGSA) only resulted in minor performance loss on DBLP-5 but caused more noticeable degradation on the Alibaba dataset. This contrast further illustrates that the model’s reliance on different modules is highly dependent on the intrinsic characteristics of the data: complex, highly dynamic, large-scale graphs necessitate the synergistic operation of the complete multi-scale attention mechanism.

5.3.2. Effect of hypergraph construction method

To further investigate the impact of different hypergraph construction methods, we compared three strategies by conducting link prediction experiments on the DBLP-3 dataset and node classification experiments on the DBLP-5 dataset under identical experimental conditions.

K-Means clustering-driven hyperedge construction models high-order relationships solely based on node attribute features; it groups nodes within the same cluster into a single hyperedge based on feature distributions, thereby capturing only attribute-level similarities while failing to explicitly characterize the intrinsic topological interactions of the original graph. K-Nearest Neighbor (K-NN) hyperedge construction focuses on static topological features by connecting each node to its K nearest neighbors; while this captures local connective proximity, it lacks a global structural perspective and tends to result in complex hypergraph structures. In contrast to these methods, the temporal-aware hypergraph construction paradigm designed for MADGN deeply integrates structural similarity metrics with temporal decay constraints. By explicitly modeling the temporal evolutionary consistency of node structural patterns, this strategy aggregates nodes that are structurally similar and exhibit consistent structural roles over recent snapshots into hyperedges, enabling the hypergraph to simultaneously capture multi-scale structural dependencies and the short-term topological continuity between adjacent snapshots.

As shown in Fig. 3, the proposed MADGN strategy achieves an AUC of 0.963 and an F1-score of 0.925 on the DBLP-3 link prediction task, and an ACC of 0.707 and an AUC of 0.898 on the DBLP-5 node classification task; all four metrics outperformed the traditional K-Means and K-NN hyperedge construction schemes. The overall performance gap exhibits a stratified pattern: in the link prediction scenario, the three methods performed similarly, with MADGN maintaining a slight lead; however, in the node classification task, performance differences were pronounced—particularly regarding accuracy (ACC), where both K-Means and K-NN achieved lower performance compared to MADGN. The comparison results demonstrate that MADGN generates multi-scale hyperedges based on temporal-aware structural similarity, enabling the flexible fusion of multi-granular high-order associations and the precise characterization of high-order structural dependencies between nodes; consequently, it achieves superior prediction and classification performance across both types of downstream tasks (see Figs. 4 and 5).

5.4. Parameter sensitivity analysis

We investigate the sensitivity of MADGN to six key hyperparameters on DBLP-3 link prediction and DBLP-5 node classification. The ranges for fine-tuned parameter settings are as follows: hidden dimension $F \in \{16, 32, 64, 128, 256\}$; negative sampling ratio $\beta \in \{1, 5, 10, 15, 20, 25\}$; number of structural attention heads H_s and temporal attention heads H_t both selected from $\{2, 4, 8, 16, 32\}$; temporal decay coefficient $\omega \in \{0, 0.25, 0.5, 1, 2, 2.5\}$; and structural similarity weight $\zeta \in \{0, 0.25, 0.5, 1, 2, 5\}$.

The hidden dimension and the numbers of attention heads mainly control model capacity. Increasing F initially improves the performance

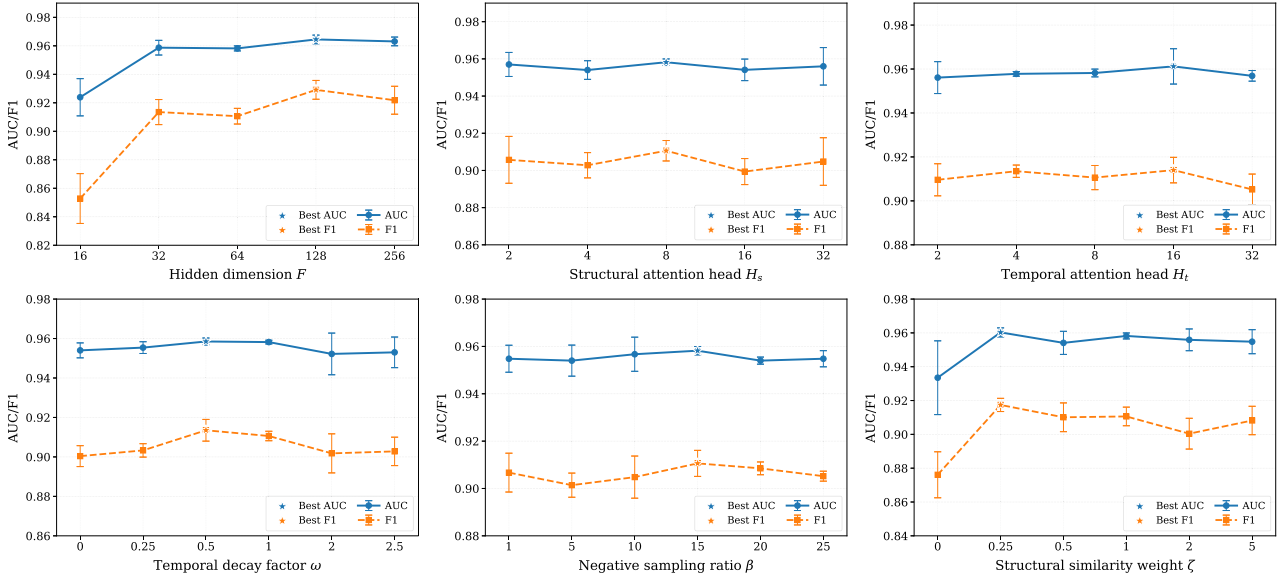


Fig. 4. Parameter sensitivity of MADGN on DBLP-3 link prediction. Curves report the mean AUC and F1 over five runs, error bars denote standard deviations, and stars mark the best values.

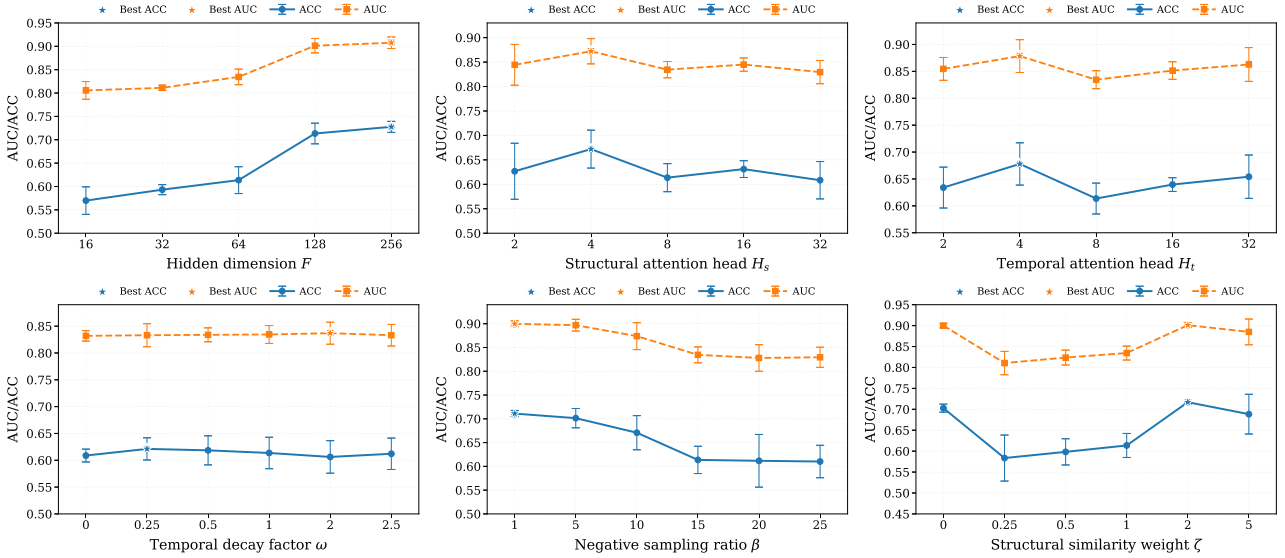


Fig. 5. Parameter sensitivity of MADGN on DBLP-5 node classification. Curves report the mean AUC and ACC over five runs, error bars denote standard deviations, and stars mark the best values.

on both tasks by providing a richer representation space, whereas further enlargement yields limited gains and may introduce redundant features. Similar trends are observed for H_s and H_t : using multiple heads improves structural and temporal aggregation, while excessively many heads reduce the feature dimension assigned to each head and provide no consistent additional benefit. These results favor moderate model dimensions and attention-head settings.

The temporal decay rate ω and negative sampling ratio β primarily affect temporal aggregation and optimization. A moderate ω provides a suitable balance between recent structural changes and historical information; very small values retain excessive outdated information, whereas large values rapidly suppress historical contributions. Performance is relatively stable over a broad range of β , although large negative-sampling ratios increase training cost without producing consistent improvements.

The structural similarity weight ζ determines the balance between connective-proximity learning and structural regularization. Setting $\zeta =$

0 generally reduces performance, indicating that structural-similarity preservation contributes useful complementary supervision. However, overly large values may overemphasize structural consistency at the expense of the downstream objective. Overall, the similar trends observed on DBLP-3 and DBLP-5 indicate that MADGN remains stable across a reasonably broad range of hyperparameter settings, while moderate values generally provide the best balance between predictive performance and computational cost.

5.5. Further analysis

Beyond downstream performance, ablation studies, and parameter sensitivity, we further analyze MADGN from the perspectives of representation quality, embedding organization, and computational efficiency. Specifically, we first evaluate current graph reconstruction and future snapshot prediction under a fixed decoder. We then examine structural-equivalence preservation and temporal smoothness, followed

Table 8

Fixed-decoder evaluation on DBLP-3 for current-graph reconstruction and future-snapshot prediction. Higher values are better.

Methods	Current graph reconstruction				Future snapshot prediction			
	AUC $\uparrow$	AP $\uparrow$	F1 $\uparrow$	ACC $\uparrow$	AUC $\uparrow$	AP $\uparrow$	F1 $\uparrow$	ACC $\uparrow$
GAEN	0.743 $\pm$ 0.018	0.613 $\pm$ 0.019	0.763 $\pm$ 0.008	0.694 $\pm$ 0.006	0.765 $\pm$ 0.005	0.698 $\pm$ 0.004	0.791 $\pm$ 0.007	0.719 $\pm$ 0.005
RTGCN	0.604 $\pm$ 0.041	0.550 $\pm$ 0.017	0.688 $\pm$ 0.024	0.557 $\pm$ 0.061	0.552 $\pm$ 0.043	0.516 $\pm$ 0.024	0.678 $\pm$ 0.015	0.540 $\pm$ 0.048
SpikeNet	0.714 $\pm$ 0.082	0.667 $\pm$ 0.064	0.729 $\pm$ 0.051	0.652 $\pm$ 0.089	0.602 $\pm$ 0.035	0.566 $\pm$ 0.027	0.675 $\pm$ 0.011	0.554 $\pm$ 0.022
DySAT	0.755 $\pm$ 0.035	0.708 $\pm$ 0.032	0.749 $\pm$ 0.015	0.689 $\pm$ 0.030	0.711 $\pm$ 0.029	0.636 $\pm$ 0.024	0.745 $\pm$ 0.014	0.676 $\pm$ 0.024
EvolveGCN	0.815 $\pm$ 0.040	0.724 $\pm$ 0.075	0.843 $\pm$ 0.030	0.823 $\pm$ 0.037	0.620 $\pm$ 0.032	0.559 $\pm$ 0.034	0.693 $\pm$ 0.019	0.593 $\pm$ 0.059
MADGN (ours)	0.912 $\pm$ 0.010	0.886 $\pm$ 0.014	0.860 $\pm$ 0.006	0.848 $\pm$ 0.005	0.919 $\pm$ 0.003	0.886 $\pm$ 0.003	0.877 $\pm$ 0.011	0.870 $\pm$ 0.009

by qualitative visualization of the learned embedding space. Finally, we compare the per-epoch training time of MADGN with representative baselines to assess its practical computational cost. For all quantitative evaluations, the same protocol and node-pair samples are used for all compared methods, and the reported results are averaged over five independent runs.

5.5.1. Fixed-decoder graph reconstruction and future prediction

We first evaluate whether the learned embeddings preserve sufficient structural information under a fixed and parameter-free decoder. Given the representations of nodes u and v at timestamp t , their connection score is computed using the inner-product decoder:

$$\hat{A}_{uv}^t = \sigma(\mathbf{o}_u^t(\mathbf{o}_v^t)^\top), \quad (21)$$

where $\sigma(\cdot)$ denotes the sigmoid function. The same decoder is applied to all methods without additional task-specific parameters.

For current graph reconstruction, the embeddings at timestamp t are used to reconstruct the observed adjacency structure A^t . For future snapshot prediction, the representations obtained from historical snapshots up to timestamp t are used to predict the adjacency structure A^{t+1} of the subsequent snapshot. The former examines whether the embeddings preserve the currently observed topology, whereas the latter evaluates whether they contain predictive information about graph evolution.

We report AUC, Average Precision (AP), F1-score, and Accuracy (ACC). AUC and AP measure the ranking quality of predicted edges, whereas F1 and ACC evaluate binary prediction performance under a decision threshold selected on the validation set. Higher values indicate better reconstruction or prediction performance.

As shown in Table 8, MADGN achieves the best results for all four metrics in current graph reconstruction. It obtains an AUC of 0.912, an AP of 0.886, an F1 of 0.860, and an ACC of 0.848. EvolveGCN is the strongest competing method in this setting, achieving an AUC of 0.815 and an AP of 0.724. The improvements in AUC and AP indicate that MADGN preserves more information about the observed graph structure under the same fixed decoder. The corresponding gains in F1 and ACC further show that this advantage is retained under the validation-selected decision threshold.

MADGN also achieves the best performance in future snapshot prediction, with an AUC of 0.919, an AP of 0.886, an F1 of 0.877, and an ACC of 0.870. In comparison, the strongest baseline in this setting, GAEN, obtains an AUC of 0.765, an AP of 0.698, an F1 of 0.791, and an ACC of 0.719. The larger performance margin in future prediction suggests that MADGN encodes not only the current adjacency structure but also transferable structural and temporal patterns that are useful for characterizing subsequent graph evolution.

Since all methods are evaluated using the same parameter-free decoder, the observed improvements cannot be attributed to a more expressive downstream predictor. Instead, they provide additional evidence that MADGN learns node representations with stronger structural reconstructability and future predictability.

5.5.2. Structural equivalence and temporal smoothness

We further evaluate whether the learned embeddings preserve structural equivalence and maintain stable temporal evolution trajectories.

Table 9

Structural-equivalence preservation and temporal smoothness on DBLP-5.

Methods	Structural AUC $\uparrow$	Structural Spearman $\uparrow$	Smoothness ratio $\downarrow$
GAEN	0.508 $\pm$ 0.002	0.014 $\pm$ 0.003	0.769 $\pm$ 0.007
RTGCN	0.517 $\pm$ 0.042	0.045 $\pm$ 0.063	0.890 $\pm$ 0.012
SpikeNet	0.509 $\pm$ 0.003	0.017 $\pm$ 0.005	0.918 $\pm$ 0.077
DySAT	0.506 $\pm$ 0.004	0.013 $\pm$ 0.006	0.629 $\pm$ 0.005
EvolveGCN	0.500 $\pm$ 0.008	0.001 $\pm$ 0.013	0.741 $\pm$ 0.032
MADGN	0.657 $\pm$ 0.081	0.273 $\pm$ 0.139	0.246 $\pm$ 0.019

All methods are evaluated on the DBLP-5 dataset using the same sets of structural node pairs and temporal snapshots.

In this paper, we employ Structural AUC and Structural Spearman correlation to jointly assess the structural equivalence of node embeddings. Structural AUC characterizes the ability of embedding similarity to distinguish between node pairs that are structurally similar and those that are structurally dissimilar; Structural Spearman correlation quantifies the rank correlation between the true structural similarity of node pairs and their similarity in the embedding space. Higher values for both metrics indicate that the embeddings better preserve structural relationships in the original network. To assess smoothness along the temporal dimension, we utilize the Smoothness Ratio, a metric that quantifies the normalized fluctuation of node embeddings between adjacent timestamps; a lower value indicates a more stable temporal evolution process for the node embeddings.

As shown in Table 9, MADGN achieves a Structural AUC of 0.657 and a Structural Spearman correlation of 0.273, both of which outperform the baseline methods. These results demonstrate that MADGN more effectively maps nodes with similar structural contexts to nearby regions in the latent space and better preserves the relative ranking of structural similarities. Furthermore, MADGN achieves the lowest Smoothness Ratio (0.246), whereas the best-performing baseline method under this metric, DySAT, records a value of 0.629. This result indicates that the representations learned by MADGN evolve more smoothly between adjacent timestamps.

It is worth noting that the evaluation of a low Smoothness Ratio should be considered alongside the model's superior performance in graph reconstruction and structural equivalence. An ineffective representation might exhibit minimal temporal variation yet fail to reconstruct the graph structure or distinguish structural roles. Therefore, the simultaneous improvement across these metrics demonstrates that the representations learned by MADGN possess both temporal stability and rich structural information.

5.5.3. Embedding visualization

To provide an intuitive view of the learned representation space, we use t-SNE to project the node embeddings into a two-dimensional space. Fig. 6 compares MADGN with five representative baselines on snapshot 6 of DBLP-3. The same labeled nodes and identical t-SNE configurations are used for all methods to ensure a consistent qualitative comparison. Different colors denote different node classes.

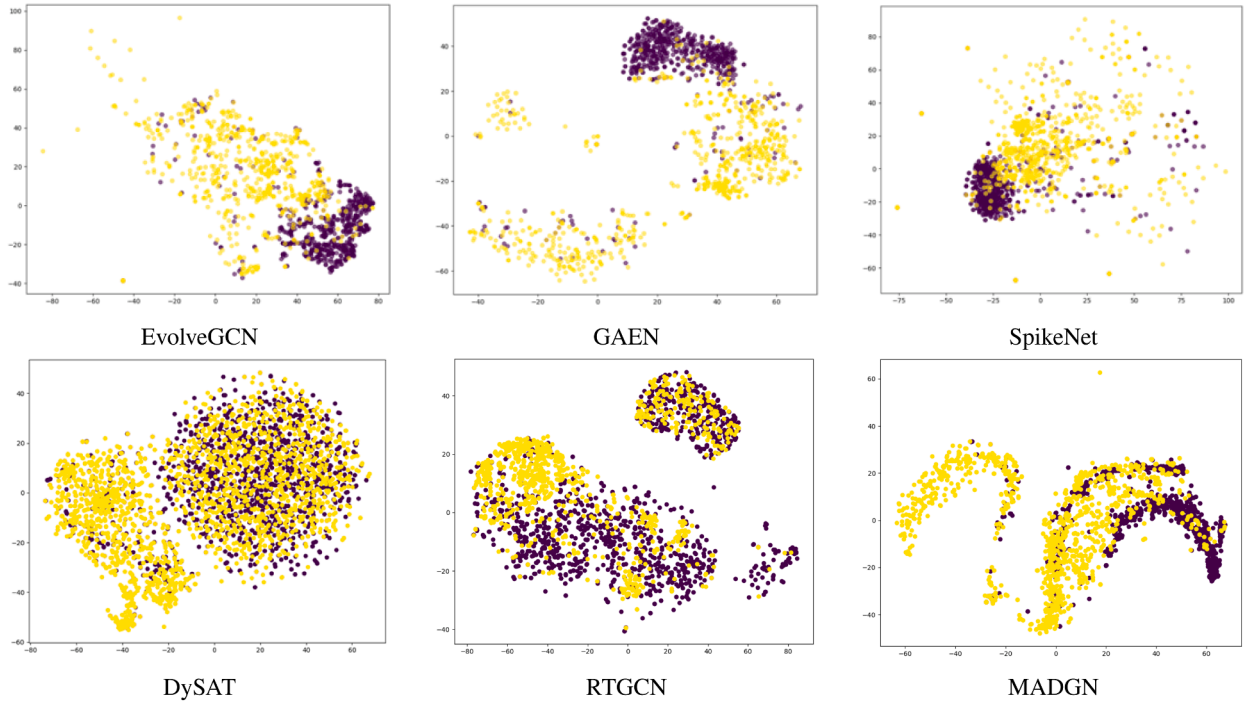


Fig. 6. Two-dimensional t-SNE projections of node embeddings from the sixth snapshot of DBLP-3. Two randomly selected classes are shown, with colors indicating class labels.

As shown in Fig. 6, the embeddings produced by several baseline methods exhibit considerable overlap between node classes or fragmentation within the same class. For example, the projections of DySAT and RTGCN contain substantial mixing between the selected classes, while the representations produced by EvolveGCN and SpikeNet form several dispersed local groups.

In contrast, the MADGN projection exhibits visually clearer inter-class separation and more compact intra-class distributions. Most nodes belonging to the same class are located in coherent regions, while nodes from different classes show less overlap. This qualitative pattern is consistent with the node classification results reported in Table 6, indicating that the representations learned by MADGN contain discriminative information useful for distinguishing node categories.

The visualization further illustrates the complementary effects of the proposed modules. STGSA captures high-order structural relations, LSA preserves local topological interactions, TFA models node-attribute evolution, and LTAF integrates these signals across extended temporal horizons. Their coordinated operation produces a representation space with more coherent structural and semantic organization. Nevertheless, t-SNE is used only as a qualitative illustration, and the main conclusions regarding representation quality are supported by the quantitative evaluations in Tables 8 and 9.

5.5.4. Computational efficiency

Finally, we evaluate the computational efficiency of MADGN by comparing its average training time per epoch with that of representative baseline methods on the DBLP-3 and Alibaba datasets. All methods were executed under identical hardware and software environments. The recorded execution times encompass forward propagation, loss calculation, backpropagation, and parameter updates, but exclude the time required for hypergraph construction.

As shown in Fig. 7, MADGN maintains competitive training efficiency across both datasets while achieving superior predictive performance. Although MADGN incorporates multiple structural and temporal representation modules, its runtime is comparable to that of representa-

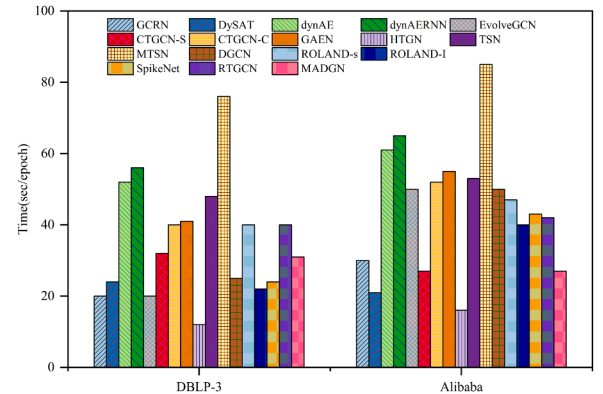


Fig. 7. Average per-epoch training time of different methods on DBLP-3 and Alibaba under the same experimental environment.

tive structure-aware and temporal graph models. This result indicates that, under the current evaluation settings, its additional multi-scale modeling capabilities do not incur excessive computational overhead.

The observed efficiency aligns with the theoretical complexity analysis. LSA performs sparse aggregation over observed graph edges, while STGSA operates based on sparse node-hyperedge associations; consequently, their computational costs scale linearly with the respective sparse structures. TFA and LTAF introduce quadratic complexity relative to the number of temporal snapshots due to their use of temporal attention mechanisms. However, given the moderate number of timestamps in the evaluation datasets, the computational overhead associated with temporal attention remains manageable in practice.

Thus, MADGN achieves a pragmatic balance between representation capability and computational cost. It effectively models local topology, high-order structural similarity, dynamic node attributes, and long-term temporal dependencies while maintaining competitive training efficiency per epoch.

6. Conclusion

Dynamic graph representation learning is essential for complex system analysis and intelligent decision-making, yet existing methods often struggle to jointly capture high-order structural information, fine-grained node features, and multi-scale temporal dependencies. To address these limitations, this paper proposes a Multi-scale Attention-based Dynamic Graph Network (MADGN), which integrates structure-aware mechanisms with hierarchical attention modeling to achieve unified learning of structural similarity and temporal dependency. By collaboratively modeling local neighborhoods, global high-order relationships, and both short-term and long-term temporal dynamics, MADGN assigns differentiated weights to structural relations and temporal states, thereby improving representation learning under sparse and evolving graph structures.

Extensive experiments on multiple real-world datasets demonstrate that MADGN consistently outperforms state-of-the-art methods on dynamic link prediction and node classification tasks, confirming the effectiveness and robustness of the proposed multi-scale attention framework. The introduction of dynamic hypergraphs provides a flexible representation for high-order structural dependencies, enhancing the model's generalization ability under evolving graph structures. Despite these advantages, MADGN still relies on hypergraph construction strategies and temporal attention mechanisms whose computational cost may increase for very long sequences or highly complex structures. Future work will investigate adaptive hypergraph construction in heterogeneous or multimodal dynamic graphs, as well as more efficient temporal modeling strategies. Overall, MADGN offers an effective and scalable solution for dynamic graph representation learning, with strong potential for large-scale real-world applications.

CRedit authorship contribution statement

Pingtao Duan: Writing – original draft, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization; **Yanqun Wu:** Writing – original draft, Visualization, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Jiayi Wei:** Visualization, Formal analysis, Data curation; **Jing Tian:** Writing – original draft, Supervision, Formal analysis; **Jiang Wu:** Visualization, Validation, Methodology; **Lin Lin:** Supervision, Resources; **Huilai Miao:** Supervision, Software, Resources; **Jinchao Cui:** Resources, Supervision; **Hui Wang:** Resources; **Dong Ouyang:** Writing – review & editing, Supervision, Resources, Funding acquisition, Conceptualization; **Bo Jin:** Supervision, Software, Resources, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by National Key R&D Program of China (2025YFC2511800), Guangdong Basic and Applied Basic Research Foundation (2026A1515011736, 2025A1515110182, 2024A1515110191), Special Project for Clinical and Basic Sci&Tech Innovation of Guangdong Medical University (GDMULCJC2025095, GDMULCJC2025111), Natural Science Foundation of China general project (82370617), and Guangdong Medical Science and Technology Research Foundation (A2024169).

References

- Arya, D., Gupta, D. K., Rudinac, S., & Worring, M. (2020). HyperSAGE: Generalizing inductive representation learning on hypergraphs. *arXiv:2010.04558*
- Bai, S., Zhang, F., & Torr, P. H. S. (2021). Hypergraph convolution and hypergraph attention. *Pattern Recognition*, 110, 107637.
- Cao, S., Lu, W., & Xu, Q. (2015). GraRep: Learning graph representations with global structural information. In *Proceedings of the 24th ACM international conference on information and knowledge management* (pp. 891–900).
- Chen, D., Zheng, S., Xu, M., Zhu, Z., & Zhao, Y. (2025). SiGNN: A spike-induced graph neural network for dynamic graph representation learning. *Pattern Recognition*, 158, 111026.
- Ding, K., Wang, J., Li, J., Li, D., & Liu, H. (2020). Be more with less: Hypergraph attention networks for inductive text classification. *arXiv:2011.00387*
- Du, H., Shi, L., Chen, X., Zhao, Y., Zhang, H., Yang, C., Zhuang, F., & Kou, G. (2024). Representation learning of temporal graphs with structural roles. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 654–665).
- Duan, P., Ren, X., & Liu, Y. (2023). Multi-relational dynamic graph representation learning. *Neurocomputing*, 558, 126688.
- Duan, P., Zhou, C., & Liu, Y. (2024). Dynamic graph representation learning via coupling-process model. *IEEE Transactions on Neural Networks and Learning Systems*, 35(9), 12383–12395. <https://doi.org/10.1109/TNNLS.2023.3257488>
- Feng, Y. et al. (2019). Hypergraph neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 3558–3565). (vol. 33).
- Gao, C., Zhu, J., Zhang, F., Wang, Z., & Li, X. (2023). A novel representation learning for dynamic graphs based on graph convolutional networks. *IEEE Transactions on Cybernetics*, 53(6), 3599–3612. <https://doi.org/10.1109/TCYB.2022.3159661>
- Goyal, P., Chhetri, S. R., & Canedo, A. (2020). dyngraph2vec: Capturing network dynamics using dynamic graph representation learning. *Knowledge-Based Systems*, 187, 104816.
- Grover, A., & Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 855–864).
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30, 1024–1034.
- Kazemi, S. M., Goel, R., Jain, K., Kobayez, I., Sethi, A., Forsyth, P., & Poupart, P. (2020). Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70), 1–73.
- Khoshraftar, S., & An, A. (2024). A survey on graph representation learning methods. *ACM Transactions on Intelligent Systems and Technology*, 15(1), 1–55.
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International conference on learning representations*.
- Li, J., Yu, Z., Zhu, Z., Chen, L., Yu, Q., Zheng, Z., Tian, S., Wu, R., & Meng, C. (2023). Scaling up dynamic graph representation learning via spiking neural networks. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 8588–8596). (vol. 37).
- Liu, J., Xu, C., Yin, C., Wu, W., & Song, Y. (2022). K-core based temporal graph convolutional network for dynamic graphs. *IEEE Transactions on Knowledge and Data Engineering*, 34(8), 3841–3853. <https://doi.org/10.1109/TKDE.2020.3033829>
- Liu, Z., Huang, C., Yu, Y., & Dong, J. (2021). Motif-preserving dynamic attributed network embedding. In *Proceedings of the web conference 2021* (pp. 1629–1638).
- Ndunguru, D. D., Xing, F., Oroni, C. Z., Ani, A., & Li, C. (2025). STG-LSTM: Spatial-temporal graph-based long short-term memory for vehicle trajectory prediction. *Multimodal Transportation*, 4(3), 100222.
- Newman, M. E. J. (2003). The structure and function of complex networks. *SIAM Review*, 45(2), 167–256.
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., & Leiserson, C. (2020). EvolveGCN: Evolving graph convolutional networks for dynamic graphs. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 5363–5370). (vol. 34).
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). DeepWalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 701–710).
- Sankar, A., Wu, Y., Gou, L., Zhang, W., & Yang, H. (2020). DySAT: Deep neural representation learning on dynamic graphs via self-attention networks. In *Proceedings of the 13th international conference on web search and data mining* (pp. 519–527).
- Seo, Y., Defferrard, M., Vandergheynst, P., & Bresson, X. (2018). Structured sequence modeling with graph convolutional recurrent networks. In *International conference on neural information processing* (pp. 362–373). Springer.
- Shi, M., Huang, Y., Zhu, X., Tang, Y., Zhuang, Y., & Liu, J. (2021). GAEN: Graph attention evolving networks. In *Proceedings of the thirtieth international joint conference on artificial intelligence* (pp. 1541–1547). <https://doi.org/10.24963/ijcai.2021/213>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in neural information processing systems* (pp. 5998–6008). (vol. 30).
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. In *International conference on learning representations*.
- Wu, Y., Fang, Y., & Liao, L. (2024). On the feasibility of simple transformer for dynamic graph modeling. In *Proceedings of the ACM web conference 2024* (pp. 870–880).

- Xu, D., Ruan, C., Körpeoglu, E., Kumar, S., & Achan, K. (2020). Inductive representation learning on temporal graphs. In *International conference on learning representations*.
- Yang, M., Zhou, M., Kalander, M., Huang, Z., & King, I. (2021). Discrete-time temporal network embedding via implicit hierarchical learning in hyperbolic space. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining* (pp. 1975–1985).
- You, J., Du, T., & Leskovec, J. (2022). ROLAND: Graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 2358–2366).
- Zhang, S., Chen, X., Xiong, Y., Wu, X., Zhang, Y., Fu, Y., Zhao, Y., & Zhang, J. (2024). Towards adaptive neighborhood for advancing temporal interaction graph modeling. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 4290–4301).
- Zheng, Y., Yi, L., & Wei, Z. (2025). A survey of dynamic graph neural networks. *Frontiers of Computer Science*, 19(6), 196323.